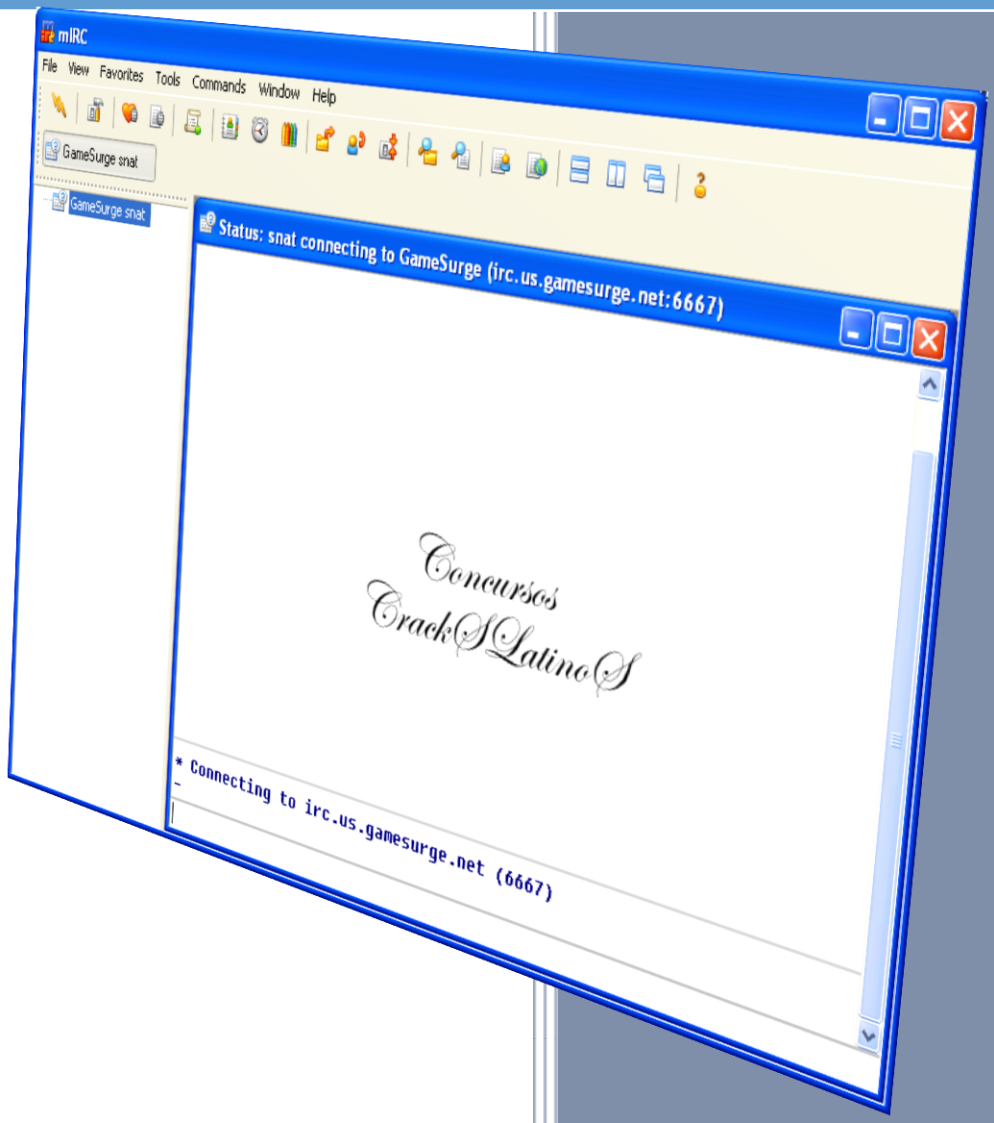
2016

Reverseando mIRC 7.43



Snat.~
[CrackSLatinoS]

Reverseando mIRC 7.43

CrackSLatinoS

Marzo 2016

Contenido

Contenido 1
1. Introducción 2
2. Material necesario 3
3. Instalando el Software 4
4. Primeras observaciones 6
5. el ASLR influye ? 7
6. Al ataque ! 9
7. Tirando de Recursos 42
8. Prueba de fuego 47
9. Notas Finales 48

1. Introducción

Hoy vamos a ver una protección muy básica consistente en un Time-Trial de 30 días, hace tiempo que no reverseo nada y me apetecía bastante hacerlo después de este tiempo. Hace un par de días estuve hablando con Juan Fary, otro integrante de CracksLatinoS acerca de este programa, en base a aPLib y algunos detalles más. La cuestión es que ni a él ni a mi nos hace falta este programa para nada, y después de escribir el tutorial borraré el crack. Nuestros fines son única y exclusivamente educativos, sé que esto parece un tópico, pero en mi caso lo digo completamente en serio, y todo aquél que me conoce sabe que esto es cierto.

Dicho esto, lo que haremos será instalar el software en Windows 7 para ver un detalle del ASLR y lo reversearemos en un Windows XP (sin ASLR) para que se vea la diferencia. Es cierto que esa protección implementada por Windows 7 (y superiores) puede despistar, pero como veremos no influye en nada en el desarrollo del proceso de nuestra sesión.

Sin más dilación pasamos al contenido.

2. Material necesario

El programa con el que trabajaremos será mIrc en su versión 7.43 descargable desde aquí:

http://www.mirc.com

Durante el tutorial utilizaremos las siguientes herramientas:

- ✓ OllyDbg 1.10 de Oleh Yuschuk
  http://www.ollydbg.de

- ✓ IDA Pro IDA Pro 6.8
  https://drive.google.com/file/d/0B13TW0I0f8O2M0FjUE12NUF4dUE/view

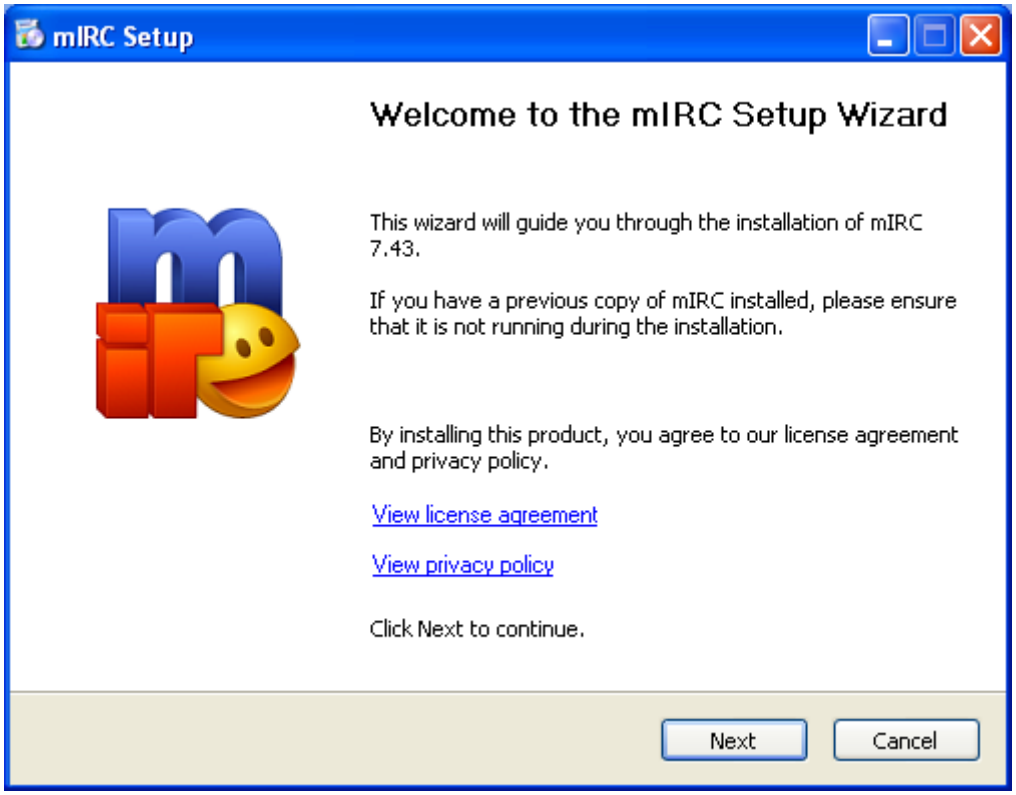
- ✓ Resource Hacker Plus
  http://download.cnet.com/Resource-Hacker/3000-2352_4-10178587.html

Herramientas descargables del web de Ricardo Narvaja:

http://www.ricardonarvaja.info/WEB/

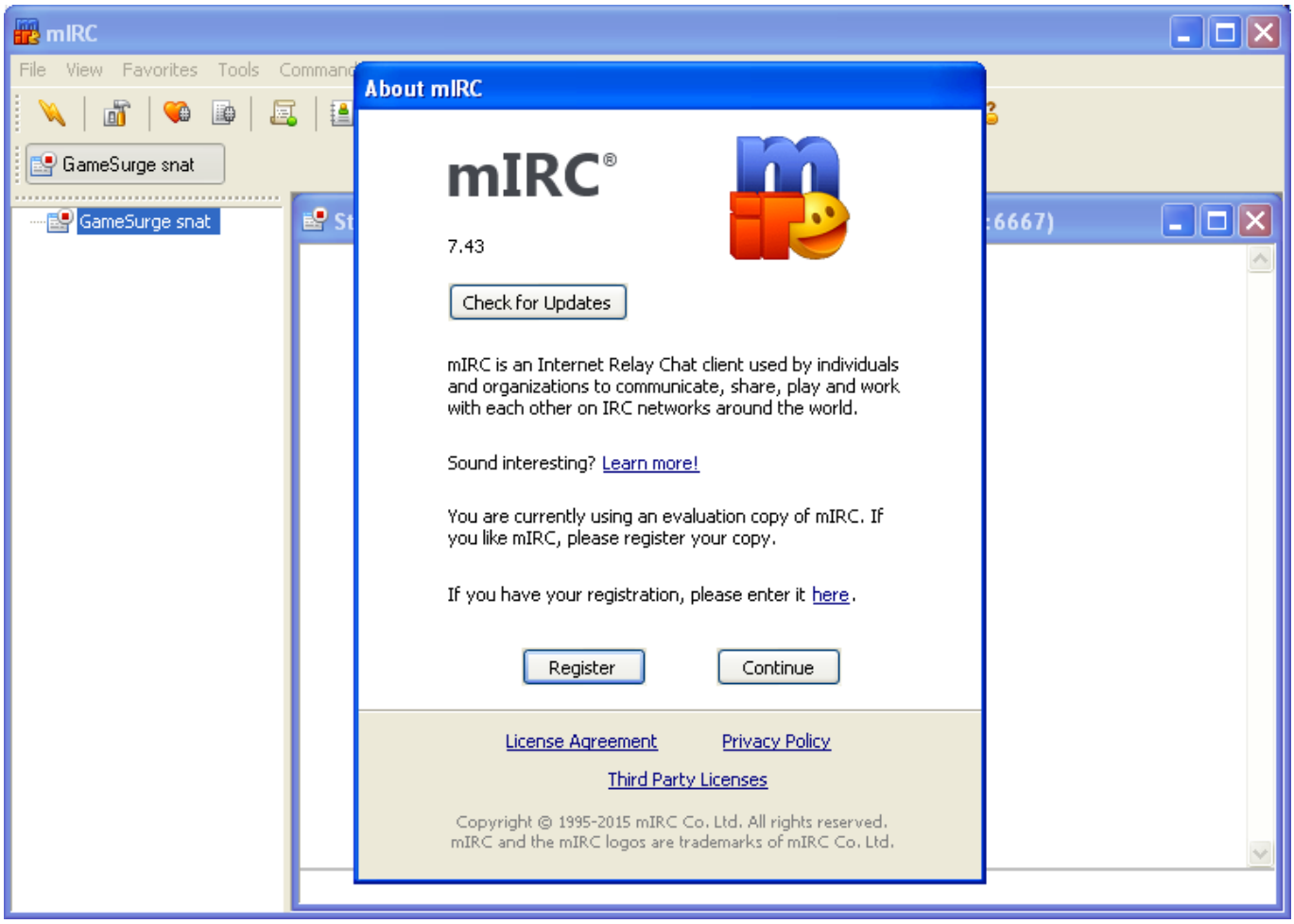
3. Instalando el Software

Este punto es completamente Trivial, realizamos una instalación Estándar:

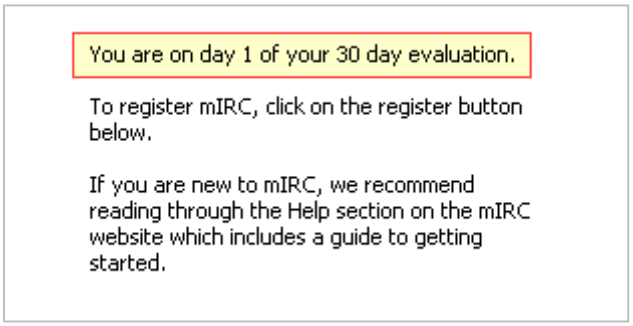


Next -> Next -> Next.. hasta completar la instalación:

Pulsamos Finish y ejecutamos el programa:



Lo primero que hace es mostrar esa molesta nagscreen que no es más que un recordatorio de que estamos en la versión trial del programa (de evaluación de 30 días), pulsamos Continue y la ventana cambia:

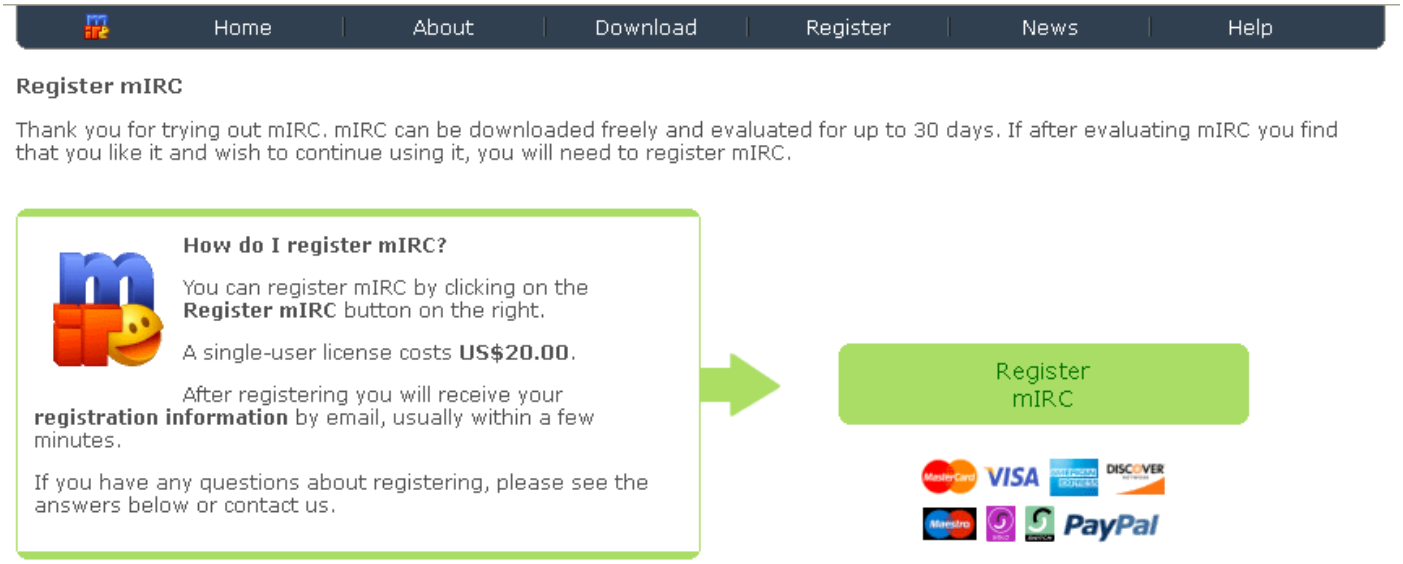


Ahora nos dice, además, el día en que nos encontramos dentro de este período de prueba (día 1).

Ya hemos visto suficiente

4. Primeras observaciones

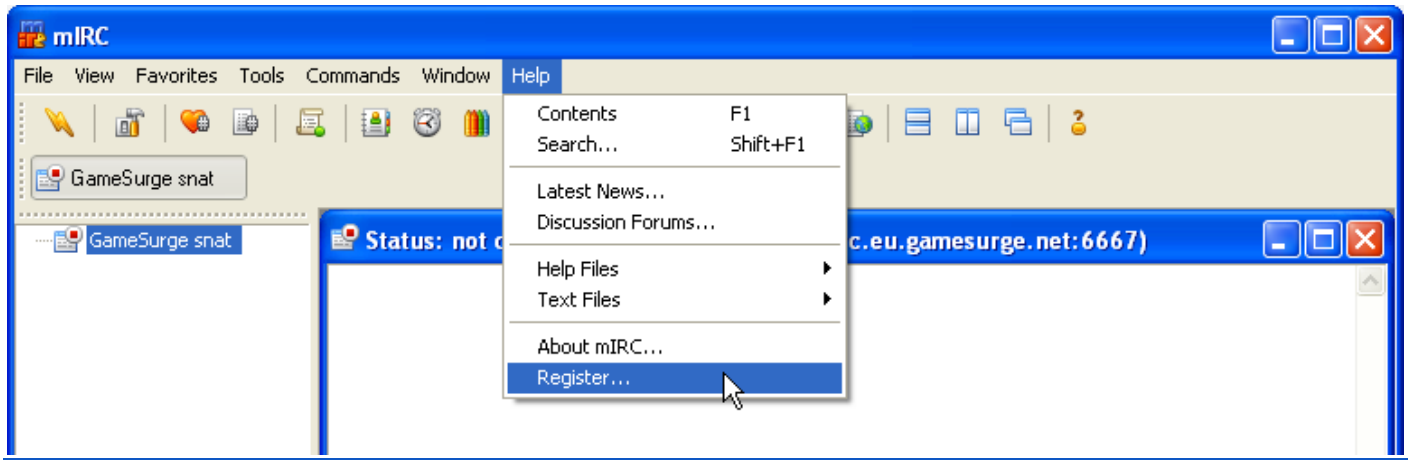
Si pulsamos el botón Register de la nagscreen nos envía a una página web del programa:



Se trata de http://www.mirc.com/register.html donde nos piden 20 pavos por el programa, en realidad no es mucho, 20 dólares al cambio serían 17 ó 18 euros, nada.

No obstante no lo vamos a comprar puesto que no lo necesitamos.

Pulsamos Continue y se cierra la nag, se abre otro cuadro de diálogo para configurar distintas opciones del programa que también cerramos, y miramos en el menú About:

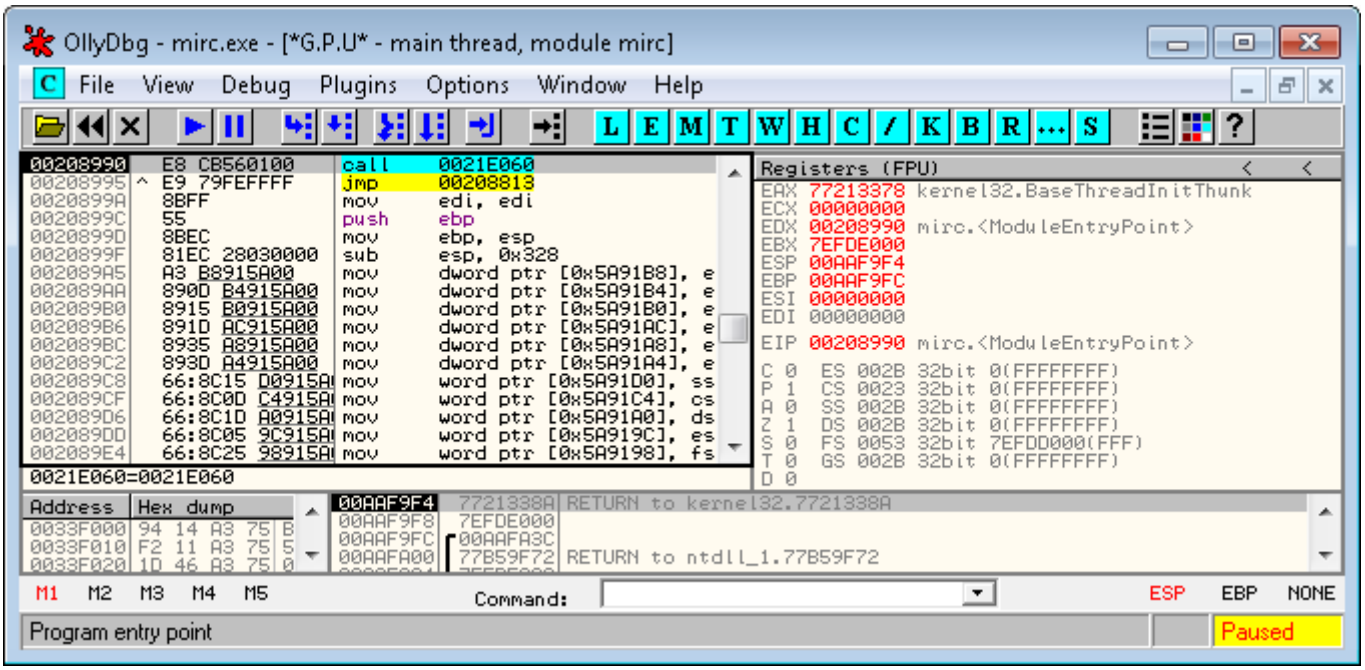


Encontramos la opción de registro, desde la cual le podemos ingresar un nombre y un serial.

Anotamos este dato y nos vamos a Windows 7.

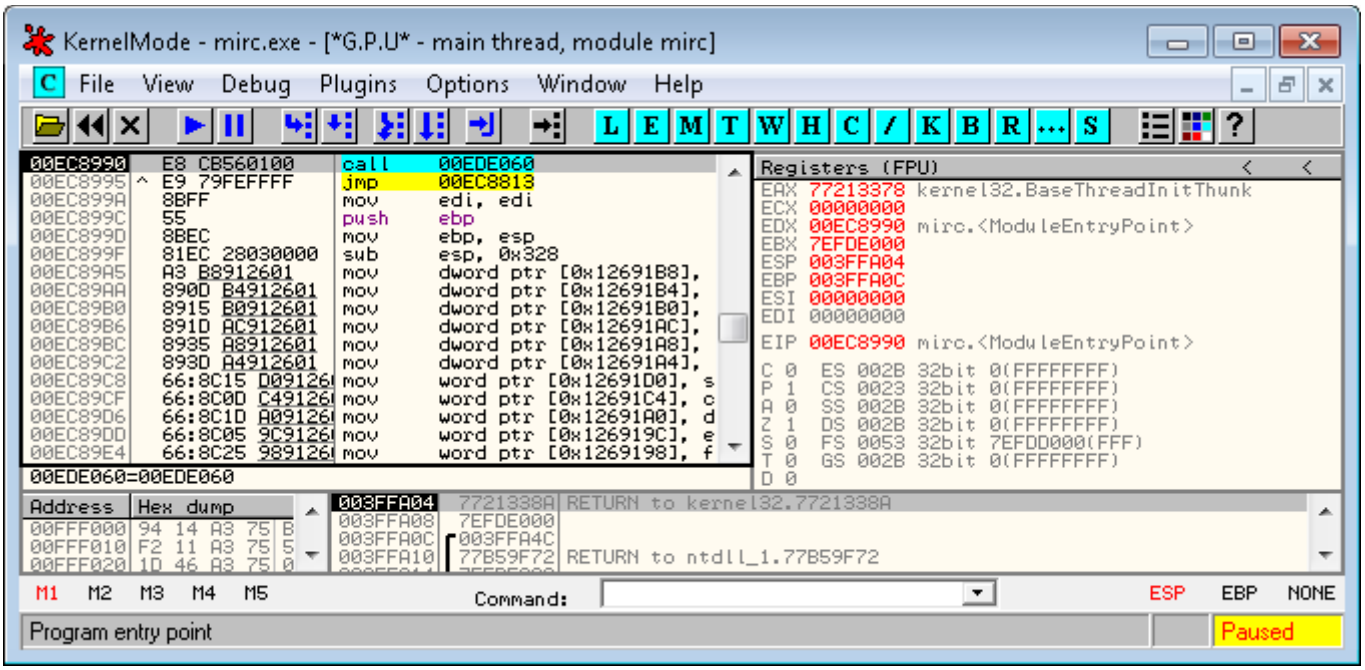
5. el ASLR influye ?

Veamos, esto es más fácil de responder con una práctica que con teoría. carguemos el ejecutable estudiado mirc.exe en Olly en Windows 7 (en este caso concreto Professional x64) y vemos esto:



Vemos que el entry point está en 0x208990.

Reiniciamos Olly varias veces con <Ctrl+F2> y volvemos a mirar:



Qué pasó ?

Ahora el Entry point no está en 0x208990 como antes sino en 0xEC8990.

Magia ? No, es cosa del ASLR, una técnica que dicho en palabras llanas consiste en reubicar el código en memoria en cada ejecución (o reinicio del sistema en según qué casos), de manera que la dirección BASE puede ser distinta, lo que acaba de ocurrir.

Si somos observadores veremos una similitud entre ambas direcciones:

```
Toma 1: 0x208990
Toma 2: 0xEC8990
```

El offset relativo no cambió, sólo lo hizo la base, en el primer caso era igual a 0x40000 y en segundo 0xD00000, por lo que en ambos casos el entry point se encuentra a una distancia relativa respecto a la base:

```
distancia: 0x1C8990
```

Según esto, el entry point del programa estará SIEMPRE en esta dirección:

```
entry point: BASE + 0x1C8990
```

Siendo BASE igual al ImageBase del binario en memoria.

Esto es precisamente lo que hace que cualquier modificación que apliquemos sobre el ejecutable será válida entre ejecución y ejecución (o entre reinicio y reinicio), pues las distancias se mantienen.

Teniendo esto en cuenta podemos afirmar, entonces, que podemos parchear el ejecutable con total normalidad y que este parche no afectará en absoluto al proceso (programa en ejecución).

Esta es la teoría, pongámoslo en práctica xDD

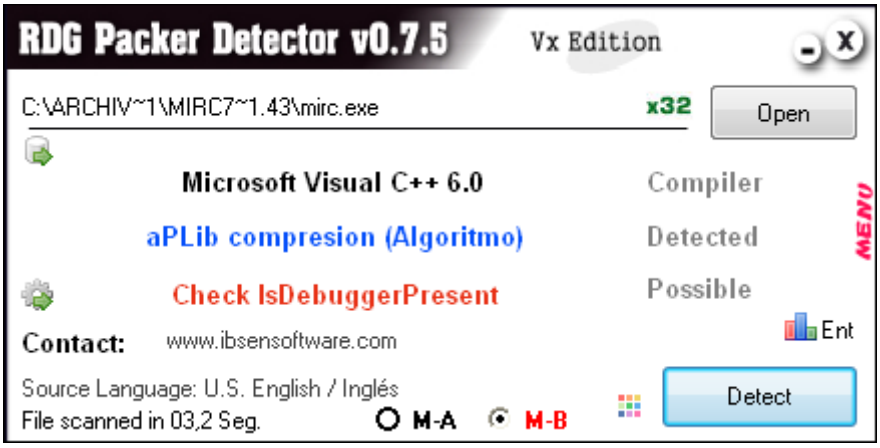
6. Al ataque !

Se acabó el palabrerío, vamos a por él !

Para el trabajo es indiferente hacerlo en Windows 7 que en XP, por comodidad lo haré en XP, de hecho para comprobar que el ASLR no nos importa probar a poner un BP común con <F2> en cualquier dirección (en Windows 7) y reiniciar varias veces Olly, veréis que aunque la dirección cambie el BP se mantiene.

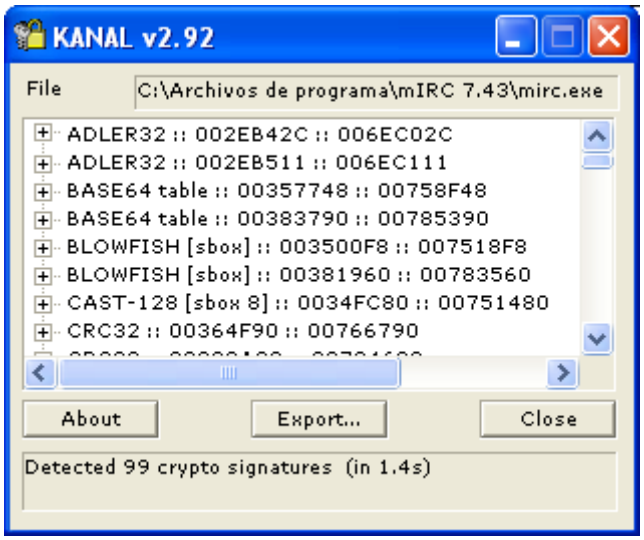
Estamos en XP, para acompañar al procedimiento voy a abrir el exe en el IDA, no es necesario, pero dado que estamos practicando lo haremos para fijarnos en detalles.

Por cierto miremos el EXE en RDG Packer a ver qué nos dice:



Visual C++ 6.0 y compresión con aPLib.

Vamos a utilizar otra herramienta (ya en desuso)e, el plugin KANAL 2.92 de PEiD 0.96:



PEiD ya no se utiliza pero yo algunas veces lo sigo utilizando, al igual que muchísimas otras tools.

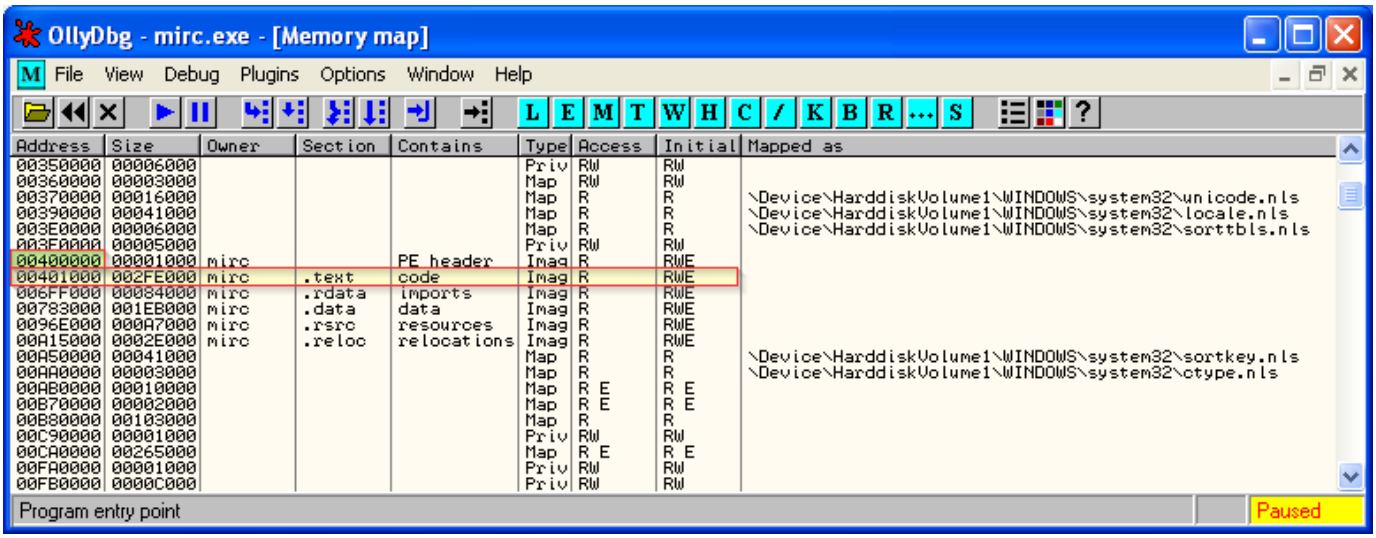
En definitiva, nos dice que se encuentran algunos algoritmos de encriptación en el ejecutable.

Esto a nosotros ahora mismo no nos importa demasiado, sólo estamos echando un vistazo, veamos las importaciones que tiene con cualquier identificador de cabeceras, DiE 0.64 por ejemplo:

Source DLL	RvaName	FirstThunk
COMDLG32.dll	0038206A	002FF080
ADVAPI32.dll	003821E0	002FF000
SHELL32.dll	003822DC	002FF4BC
ole32.dll	003823C6	002FF910
OLEAUT32.dll	003823D0	002FF478
CRYPT32.dll	00382486	002FF098

Entre ellas nos llama la atención ese ctypt32, sigamos..

Cargamos el exe en OllyDbg y miremos las secciones en memoria:



El imageBase está en 0x400000, el entry point sabemos que estará a una distancia de 0x1C8990:

```
005C8990  $  E8 CB560100   call   005DE060                    // ** entry point
005C8995  .^ E9 79FEFFFF   jmp    005C8813
005C899A  >  8BFF          mov    edi, edi
005C899C  .  55            push   ebp
005C899D  .  8BEC          mov    ebp, esp
005C899F  .  81EC 2803000( sub    esp, 0x328
005C89A5  .  A3 B8919600   mov    dword ptr [0x9691B8], eax
005C89AA  .  890D B491960( mov    dword ptr [0x9691B4], ecx
005C89B0  .  8915 B091960( mov    dword ptr [0x9691B0], edx
005C89B6  .  891D AC91960( mov    dword ptr [0x9691AC], ebx
005C89BC  .  8935 A891960( mov    dword ptr [0x9691A8], esi
005C89C2  .  893D A491960( mov    dword ptr [0x9691A4], edi
```

Efectivamente, está dónde debería:

```
ep = ImageBase + 0x1C8990
   = 0x400000 + 0x1C8990
   = 0x5C8990
```

Lo que demuestra la teoría.

Y ahora así vamos a atacarlo !

Dejamos correr el programa con <F9> para que arranque y nos muestre la nagscreen inicial. Cerramos la NAG y el diálogo de Opciones, la ventana principal queda visible en el escritorio.

Vamos al menú Help -> Register.. y lo rellenamos con cualquier valor:

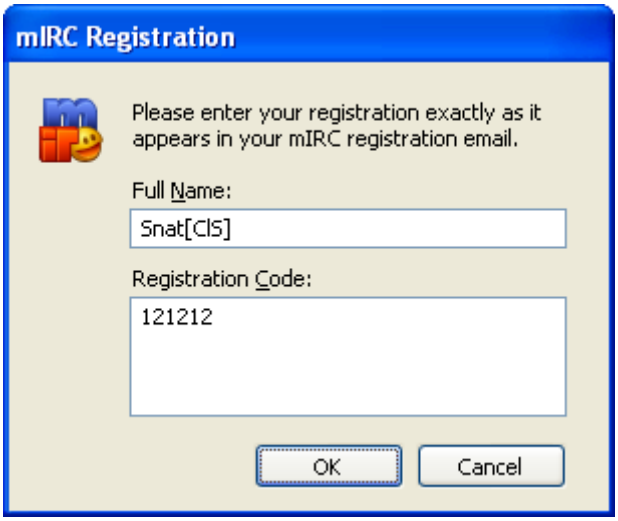
```
Name: Snat[ClS]
Serial: 121212
```

Como en los viejos tiempos, aceptamos y tarascazo:

Pulsamos <Ok> y nos da otra oportunidad.

Antes de intentarlo de nuevo colocamos los BPs más comunes para interceptar la captura de los valores de los campos de texto name y serial, lo haremos en las funciones GetWindowTextA, GetWindowTextW GetDlgItemTextA y GetDlgItemTextW.

Ahora sí, probamos otra vez, pero el BP de GetWindowTextW molesta porque para continuamente, lo deshabilitamos pero dejando los otros tres puestos, lo volvemos a intentar:



Pulsamos <OK> y nada, los BP's no paran, nos saca el cartelito anterior directamente.

Cambiaremos de estrategia, está claro que por aquí no le pillamos.

Repetimos el proceso anterior para forzar a que nos muestre el cartel de error "Your registration does not appear to be valid..", pero esta vez antes de aceptarlo colocamos un BP en la función DestroyWindow, hecho esto ahora sí cerramos el mensaje de error <OK> y Olly para en el BP que acabamos de poner (en la api DestroyWindow), lo quitamos y vamos traceando con <F8>, estamos en user32.dll, vamos pasando RETs hasta llegar al código del ejecutable mirc.exe (origen de la llamada), es aquí:

```
0047F421  .  83F8 FF        cmp     eax, -0x1
0047F424  .v 75 13          jnz     short 0047F439
0047F426  >  A1 2C7E9500    mov     eax, dword ptr [0x957E2C]
0047F42B  .  57             push    edi                          lParam
0047F42C  .  53             push    ebx                          DlgProc
0047F42D  .  0FB7D5         movzx   edx, bp
0047F430  .  56             push    esi                          hOwner
0047F431  .  52             push    edx                          pTemplate
0047F432  .  50             push    eax                          hInst => 00400000
0047F433  .  FF15 98F56F0(  call    dword ptr [<&USER32.DialogBoxPa  DialogBoxParamW
0047F439  >  5F             pop     edi                          mirc.007CB6E8
0047F43A  .  5E             pop     esi
0047F43B  .  5D             pop     ebp
Stack [0020A6D4]=007CB6E8 (mirc.007CB6E8), UNICODE "mIRC Registration"
edi=0020A708, (UNICODE "mIRC Registration")
Jump from 0047F424
```

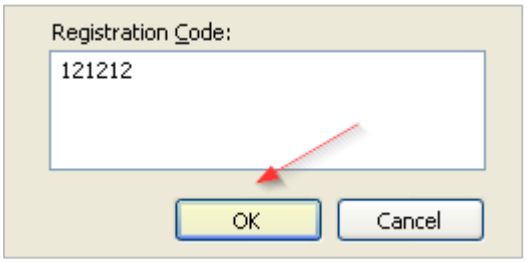
Venimos del call a DialogBoxParamA. Lo que queremos saber es cómo hemos llegado hasta aquí, es decir, el camino que hemos recorrido, si vamos traceando a la inversa con <F8> salimos aquí:

```
00501FD5  .  56             push    esi
00501FD6  .  E8 A593FEFF    call    004EB380                     // ** Call Padre
00501FDB  .  6A 02          push    0x2
00501FDD  .  68 D0200000    push    0x20D0
00501FE2  .  68 90CD8100    push    0081CD90
00501FE7  .  E8 C4D80900    call    0059F8B0
00501FEC  .  6A 02          push    0x2
00501FEE  .  68 D0200000    push    0x20D0
00501FF3  .  68 60EE8100    push    0081EE60
00501FF8  .  E8 B3D80900    call    0059F8B0
00501FFD  .  83C4 40        add     esp, 0x40
00502000  .  68 10A40000    push    0xA410
00502005  .  53             push    ebx
00502006  .  68 300F8200    push    00820F30
0050200B  .  E8 70D30B00    call    005BF380
00502010  .  6A 10          push    0x10
00502012  .  68 08C57000    push    0070C508                     UNICODE "UserName"
00502017  .  68 01000080    push    0x80000001
0050201C  .  E8 5FE5FFFF    call    00500580
00502021  .  6A 10          push    0x10
00502023  .  68 F8C47000    push    0070C4F8                     UNICODE "License"
00502028  .  68 01000080    push    0x80000001
0050202D  .  E8 4EE5FFFF    call    00500580
00502032  .  6A 50          push    0x50
00502034  .  68 E4C47000    push    0070C4E4                     UNICODE "Validated"
00502039  .  68 01000080    push    0x80000001
0050203E  .  E8 3DE5FFFF    call    00500580
00502043  .  A1 B04C7900    mov     eax, dword ptr [0x794CB0]
```

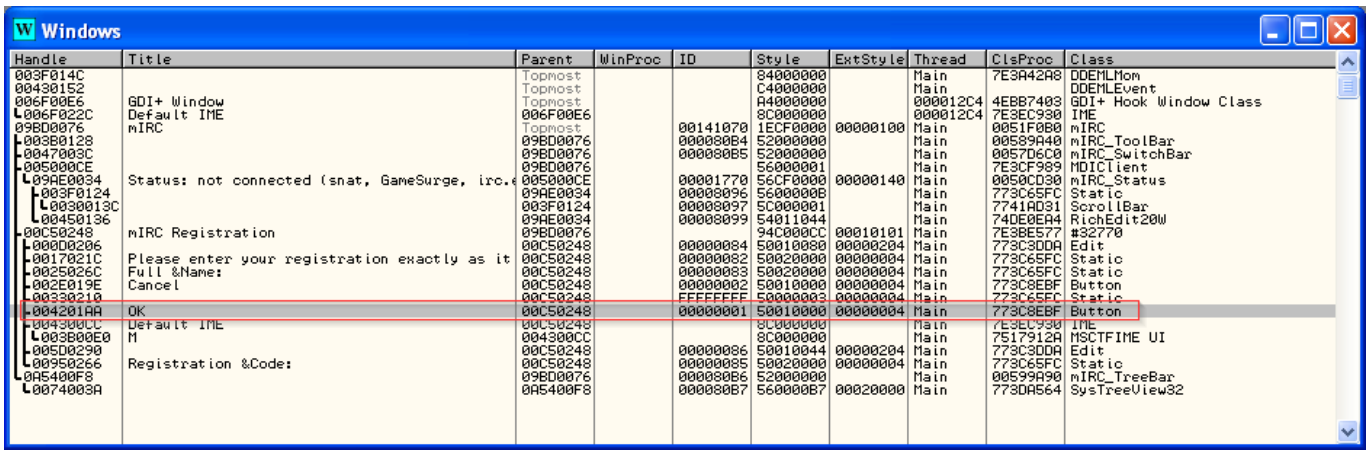
Si miramos por encima vemos llamadas a KillTimer, PostMessageW, SendDlgItemMessageW y muchísimos saltos condicionales que no aportan nada, no vemos ninguna api que por ahora nos ayude. Si seguimos los retornos caeremos en User32.dll, comctl32.dll, otra vez user, de nuevo comctl, etc, etc.. no hay salida.

Al final queda loopeando en USer32.dll entre las apis PeekMessageW, IsDialogMessageW, WaitMessageA, TranslateMessage y DispatchMessageW, de aquí no sale, quedando a la espera de procesar Mensajes (los que se le envían a las ventanas).

Lo que haremos será tratar de encontrar el procedimiento encargado de gestionar la pulsación del botón <OK> de la ventanita de registro:



Necesitamos el manejador del botón (que en realidad es una ventana), éste lo podemos encontrar en la ventanita de Windows de Olly (con la letra W), entramos en ella y buscamos el botón <OK>:



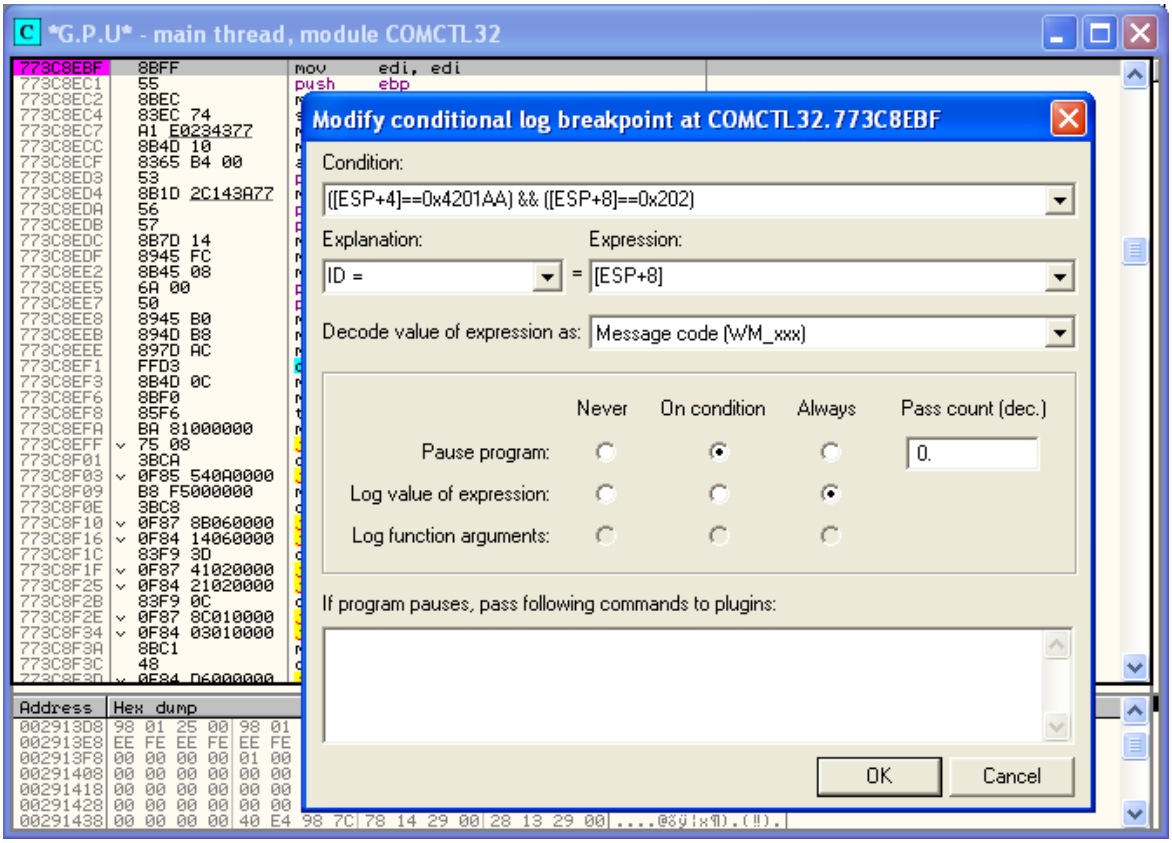
Anotamos el Handle:

```
Handle <OK> : 0x4201AA
```

Hay que tener en cuenta que este valor muy probablemente cambiará si reiniciamos Olly.

Ahora necesitamos el ID del control WM_LBUTTONUP, es decir, cuando soltamos el ratón cuando se pulsa éste y se activa el procedimiento, este ID es el 0x202, una forma de buscarlo para el que no tenga inet es colocando un BP Condicional Log en el ClassProc del botón y logguear todos los mensajes, así se descubre.

Pulsamos con el botón derecho del ratón sobre el botón <OK> en la ventana 'Windows' en la que estamos y elegimos 'Follow ClassProc', y colocamos un BP Conditional Log:



Por qué así ?

Si colocamos un BP en el ClassProc con <F2> y miramos la pila cuando para vemos porqué:

```
ESP ==>   7E398734  RETURN to USER32.7E398734
ESP+4     003F0F50  Handle Boton
ESP+8     00000081  ID Mensaje
ESP+C     00000000
ESP+10    0020A090
ESP+14    773C8EBF  COMCTL32.773C8EBF
ESP+18    DCBAABCD
ESP+1C    00000000
ESP+20    0020A090
```

Ahí está, en [ESP+4] tenemos el Handle de la ventana (del botón <OK>) y en [ESP+8] el ID del mensaje. Esta captura es de muestra y no son los valores de nuestro botón <OK> que tiene handle igual a 0x4201AA.

Hemos puesto como 'Decode value of expression as:' el valor Message code (WM_xx) y que es precisamente lo que estamos esperando.

Y ahora así, con el BreakPoint Condicional puesto en el ClassProc volvemos a la ventana de registro y pulsamos el botón <OK> con el ratón, al soltarlo Olly para en el BP:

```
*G.P.U* - main thread, module COMCTL32
773C8EBF  8BFF          mov   edi, edi
773C8EC1  55            push  ebp
773C8EC2  8BEC          mov   ebp, esp
773C8EC4  83EC 74       sub   esp, 0x74
773C8EC7  A1 E0234377   mov   eax, dword ptr [0x774323E0]
773C8ECC  8B4D 10       mov   ecx, dword ptr [ebp+0x10]
773C8ECF  8365 B4 00    and   dword ptr [ebp-0x4C], 0x0
773C8ED3  53            push  ebx
773C8ED4  8B1D 2C143A77 mov   ebx, dword ptr [<&USER32.GetWin  USER32.GetWindowLon
773C8EDA  56            push  esi
773C8EDB  57            push  edi
773C8EDC  8B7D 14       mov   edi, dword ptr [ebp+0x14]
773C8EDF  8945 FC       mov   dword ptr [ebp-0x4], eax
773C8EE2  8B45 08       mov   eax, dword ptr [ebp+0x8]
773C8EE5  6A 00         push  0x0

Registers (FPU)
EAX 7FFDE000
ECX 40000000
EDX 00000040
EBX 00000000
ESP 00216D48
EBP 00216D70
ESI 773C8EBF COMCTL32.773C8E
EDI 00216DAC
EIP 773C8EBF COMCTL32.773C8E
C 0  ES 0023 32bit 0(FFFFFFF
P 0  CS 001B 32bit 0(FFFFFFF
A 0  SS 0023 32bit 0(FFFFFFF
Z 0  DS 0023 32bit 0(FFFFFFF

Address   Hex dump                ASCII
002913D8  98 01 25 00 98 01 25 00 ÿ0%.ÿ0%.
002913E0  EE FE EE FE EE FE EE FE
002913E8  EE FE EE FE EE FE EE FE
002913F0  08 00 04 00 B5 07 18 00

ESP ==>   7E398734  RETURN to USER32.7E398734
ESP+4     004201AA  mirc.004201AA
ESP+8     00000202
ESP+C     00000000
ESP+10    000A002E
ESP+14    773C8EBF  COMCTL32.773C8EBF
```

Bien, la cosa funciona, lo siguiente es localizar el procedimiento que gestiona la pulsación del botón, este código, lógicamente, debe residir en el código del ejecutable mirc.exe, que deberá actuar en consecuencia, es decir, deberá validar el name y el serial puestos y que es lo que nosotros queremos investigar.

Colocamos un BPM on Access en la sección de código del ejecutable:

```
Memory map
Address   Size     Owner  Section  Contains   Type  Access  Initial  Mapped as
00400000  00001000 mirc            PE header  Imag  R       RWE
00401000  002FE000 mirc   .text    code
006FF000  00084000 mirc   .rdata   import
00783000  001EB000 mirc   .data    data
0096E000  000A7000 mirc   .rsrc    resour
00A15000  0002E000 mirc   .reloc   reloca
00A50000  00041000                                          iskVolume1\WINDOWS\s
00AA0000  00003000                                          iskVolume1\WINDOWS\s
00AB0000  00010000
00B70000  00002000
00B80000  00103000
00C90000  00001000
00CA0000  00271000
00FA0000  00001000
00FB0000  0000C000
00FC0000  00002000
00FD0000  00007000
01010000  00001000
01090000  00010000

Actualize
View in Disassembler        Enter
Dump in CPU
Dump
Search                      Ctrl+B
Set break-on-access         F2
Set memory breakpoint on access
Set memory breakpoint on write
```

Pulsamos <F9> y caemos aquí:

```
00501D80  .   B8 A4410000  mov     eax, 0x41A4                      // ** rompe BPM
00501D85  .   E8 C6C90B00  call    005BE750
00501D8A  .   A1 888A7800  mov     eax, dword ptr [0x788A88]
00501D8F  .   33C4         xor     eax, esp
00501D91  .   898424 A0410 mov     dword ptr [esp+0x41A0], eax
00501D98  .   8B8424 AC410 mov     eax, dword ptr [esp+0x41AC]
00501D9F  .   53           push    ebx
00501DA0  .   55           push    ebp
00501DA1  .   56           push    esi
00501DA2  .   8BB424 B4410 mov     esi, dword ptr [esp+0x41B4]
00501DA9  .   57           push    edi
00501DAA  .   3D 10010000  cmp     eax, 0x110                       Switch (cases 2..111)
00501DAF  .v  0F87 2901000 ja      00501EDE
00501DB5  .v  74 5C        je      short 00501E13
00501DB7  .   83E8 02      sub     eax, 0x2
00501DBA  .v  74 25        je      short 00501DE1
00501DBC  .   83E8 04      sub     eax, 0x4
00501DBF  .v  0F85 F00C000 jnz     00502AB5
00501DC5  .   398424 C0410 cmp     dword ptr [esp+0x41C0], eax      Case 6 (WM_ACTIVATE) of swi
00501DCC  .v  0F84 E30C000 je      00502AB5
00501DD2  .   C705 E453780 mov     dword ptr [0x7853E4], 0x1CC
00501DDC  .v  E9 D40C0000  jmp     00502AB5
00501DE1  >   A1 3C1E8300  mov     eax, dword ptr [0x831E3C]        Case 2 (WM_DESTROY) of swit
00501DE6  .   33DB         xor     ebx, ebx
00501DE8  .   3BC3         cmp     eax, ebx
00501DEA  .v  74 09        je      short 00501DF5
00501DEC  .   50           push    eax
00501DED  .   E8 48D60B00  call    005BF43A
```

Ahí se ven los condicionales WM_xx (ACTIVATE, DESTROY, etc..) nosotros venimos con WM_LBUTTONUP por lo que traceando el código nos lleva directamente a esta zona:

```
00502780  .   68 9CFA6F00  push    006FFA9C
00502785  .   68 9CFA6F00  push    006FFA9C
0050278A  .   68 9CFA6F00  push    006FFA9C
0050278F  .   56           push    esi
00502790  .   E8 BBF2FFFF  call    00501A50
00502795  .   83C4 14      add     esp, 0x14
00502798  .   8BC5         mov     eax, ebp
0050279A  .v  E9 18030000  jmp     00502AB7
0050279F  >   55           push    ebp
005027A0  .   56           push    esi
005027A1  .   E8 BAD6FFFF  call    004FFE60
005027A6  .   8B3D 28F86F0 mov     edi, dword ptr [<&USER32.SendD   USER32.SendDlgItemMessageW
005027AC  .   83C4 08      add     esp, 0x8
005027AF  .   68 08F58200  push    0082F508                         lParam = 0x82F508
005027B4  .   68 00010000  push    0x100                            wParam = 0x100
005027B9  .   6A 0D        push    0xD                              Message = WM_GETTEXT
005027BB  .   68 84000000  push    0x84                             ControlID = 84 (132.)
005027C0  .   56           push    esi                              hWnd
005027C1  .   FFD7         call    edi                              SendDlgItemMessageW
005027C3  .   68 38D48200  push    0082D438                         lParam = 0x82D438
005027C8  .   68 00010000  push    0x100                            wParam = 0x100
005027CD  .   6A 0D        push    0xD                              Message = WM_GETTEXT
005027CF  .   68 86000000  push    0x86                             ControlID = 86 (134.)
005027D4  .   56           push    esi                              hWnd
005027D5  .   FFD7         call    edi                              SendDlgItemMessageW
005027D7  .   6A 20        push    0x20
005027D9  .   68 08F58200  push    0082F508                         UNICODE "Snat[ClS]"
005027DE  .   E8 BD530700  call    00577BA0
```

Y acabamos descubriendo la rutina de respuesta a la pulsación del botón <OK>, se ven dos llamadas a la api SendDlgItemMessageW con un Message igual a WM_GETTEXT, que lo que hará será dejar el contenido de los EditBox (name y serial) en sendos búfers 0x82F508 y 0x82D438 respectivamente.

A partir de aquí somos libres de investigar lo que hace con el serial, quitamos TODOS los BP's, BPM's y BPCL's que hayamos puesto y dejamos únicamente un BP normal con <F2> en 0x50279F para romper aquí cada vez que lo necesitemos.

Pasamos las llamadas a SendDlgItemMessageW y vemos los datos recogidos en el dump:

```
Address   Hex dump                                          ASCII
0082F508  53 00 6E 00 61 00 74 00 5B 00 43 00 6C 00 53 00  S.n.a.t.[.C.l.S.
0082F518  5D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ].............
0082F528  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082F538  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082F548  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082F558  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082F568  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082F578  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Lo mismo para el serial:

```
Address   Hex dump                                          ASCII
0082D438  31 00 32 00 31 00 32 00 31 00 32 00 00 00 33 00  1.2.1.2.1.2...3.
0082D448  32 00 32 00 33 00 00 00 00 00 00 00 00 00 00 00  2.2.3...........
0082D458  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082D468  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082D478  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082D488  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082D498  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082D4A8  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Lo más difícil ya lo hemos hecho.

Un poco más de traceo nos lleva a localizar los puntos calientes de comprobación del serial:

```
0050281D  .  68 38D48200   push   0082D438                    UNICODE "121212"
00502822  .  68 08F58200   push   0082F508                    UNICODE "Snat[ClS]"
00502827  .  E8 E4E2FFFF   call   00500B10                    // ** comprueba serial
0050282C  .  83C4 08       add    esp, 0x8
0050282F  .  85C0          test   eax, eax
00502831  .v 0F84 0D01000( je     00502944
00502837  .  68 68B38200   push   0082B368
0050283C  .  68 38D48200   push   0082D438                    UNICODE "121212"
00502841  .  E8 DAE4FFFF   call   00500D20
00502846  .  A1 58B38200   mov    eax, dword ptr [0x82B358]
0050284B  .  83C4 08       add    esp, 0x8
```

Ese call de 0x502827 hace las verificaciones oportunas:

```
00500B8B  .  8D9424 AC080(  lea    edx, dword ptr [esp+0x8AC]
00500B92  .  52             push   edx
00500B93  .  E8 18FDFFFF    call   005008B0                    // ** comprueba serial
00500B98  .  8B8C24 14110(  mov    ecx, dword ptr [esp+0x1114]
00500B9F  .  83C4 44        add    esp, 0x44
00500BA2  .  F7D8           neg    eax
00500BA4  .  5F             pop    edi
00500BA5  .  1BC0           sbb    eax, eax
00500BA7  .  5E             pop    esi
00500BA8  .  33CC           xor    ecx, esp
00500BAA  .  F7D8           neg    eax                         // ** debe ser EAX==0x1
00500BAC  .  E8 83DB0B00    call   005BE734
00500BB1  .  81C4 CC10000(  add    esp, 0x10CC
00500BB7  .  C3             ret
```

Ese subcall es el encargado, reseñable hay esto:

```
005009E9  .  896C24 10      mov    dword ptr [esp+0x10], ebp
005009ED  .  8D78 02        lea    edi, dword ptr [eax+0x2]
005009F0  >  66:8B10        mov    dx, word ptr [eax]
005009F3  .  83C0 02        add    eax, 0x2
005009F6  .  66:3BD3        cmp    dx, bx
005009F9  .^ 75 F5          jnz    short 005009F0
005009FB  .  2BC7           sub    eax, edi
005009FD  .  D1F8           sar    eax, 1
005009FF  .  BA 03000000    mov    edx, 0x3
00500A04  .  3BC2           cmp    eax, edx                    // ** length( Name )  < 3
00500A06  .v 7E 24          jle    short 00500A2C
00500A08  .v EB 06          jmp    short 00500A10
edx=00000003 Longitud minima
eax=00000009 Longitud nombre
```

El nombre debe tener una longitud mínima de 4 caracteres, con el que genera un número:

```
00500A10  >  0FB73C56       movzx  edi, word ptr [esi+edx*2]
00500A14  .  0FAF7C8C 14    imul   edi, dword ptr [esp+ecx*4+0x14]
00500A19  .  41             inc    ecx
00500A1A  .  03EF           add    ebp, edi
00500A1C  .  83F9 26        cmp    ecx, 0x26
00500A1F  .v 7E 02          jle    short 00500A23
00500A21  .  33C9           xor    ecx, ecx
00500A23  >  42             inc    edx
00500A24  .  3BD0           cmp    edx, eax
00500A26  .^ 7C E8          jl     short 00500A10
00500A28  .  896C24 10      mov    dword ptr [esp+0x10], ebp   // ** genera 0x199B
00500A2C  >  8BC6           mov    eax, esi
```

En este caso quedó EBP==0x0000199B

Y llega a otro loop:

```
00500A50  >  0FB77C4E FE  movzx   edi, word ptr [esi+ecx*2-0x2]
00500A55  .  0FB72C4E     movzx   ebp, word ptr [esi+ecx*2]
00500A59  .  0FAFFD       imul    edi, ebp
00500A5C  .  0FAF7C94 14  imul    edi, dword ptr [esp+edx*4+0x14]
00500A61  .  42           inc     edx
00500A62  .  03DF         add     ebx, edi
00500A64  .  83FA 26      cmp     edx, 0x26
00500A67  .v 7E 02        jle     short 00500A6B
00500A69  .  33D2         xor     edx, edx
00500A6B  >  41           inc     ecx
00500A6C  .  3BC8         cmp     ecx, eax
00500A6E  .^ 7C E0        jl      short 00500A50
00500A70  .  8B6C24 10    mov     ebp, dword ptr [esp+0x10]     // ** genera 0x90D47
00500A74  >  83BC24 C0000 cmp     dword ptr [esp+0xC0], 0x0
```

Donde deja EBX==0x00090D47, ambos valores los obtuvo a partir del nombre.

A continuación llega a un call al que le pasa un puntero al serial introducido y el valor 0x2D:

```
00500A9D  .  5B            pop     ebx
00500A9E  .  81C4 A400000  add     esp, 0xA4
00500AA4  .  C3            ret
00500AA5  >  8BBC24 BC000  mov     edi, dword ptr [esp+0xBC]
00500AAC  .  6A 2D         push    0x2D                  Arg2 = 0000002D
00500AAE  .  57            push    edi                   Arg1 = 00211804
00500AAF  .  E8 7ADD0B00   call    005BE82E              mirc.005BE82E
00500AB4  .  8BF0          mov     esi, eax
00500AB6  .  83C4 08       add     esp, 0x8
00500AB9  .  85F6          test    esi, esi
00500ABB  .^ 0F84 15FFFFF  je      005009D6
edi=00211804, (UNICODE "121212")
```

Y lo que hace es buscar un guion en el serial. Este check no lo pasamos, cambiaremos el serial:

```
Nombre: Snat[ClS]
Serial: 1212-3434
```

Y Lo volvemos a probar:

```
00500AA5  >  8BBC24 BC000  mov     edi, dword ptr [esp+0xBC]
00500AAC  .  6A 2D         push    0x2D                  Arg2 = 0000002D
00500AAE  .  57            push    edi                   Arg1
00500AAF  .  E8 7ADD0B00   call    005BE82E              mirc.005BE82E
00500AB4  .  8BF0          mov     esi, eax
00500AB6  .  83C4 08       add     esp, 0x8
00500AB9  .  85F6          test    esi, esi
00500ABB  .^ 0F84 15FFFFF  je      005009D6              // ** no guion
00500AC1  .  33C9          xor     ecx, ecx
00500AC3  .  57            push    edi                   Arg1
00500AC4  .  66:890E       mov     word ptr [esi], cx
```

Encuentra el guion que busca y el JE que antes se producía ahora no lo hace y llega a otro call donde pasa a hexadecimal el valor de la primera parte del serial (antes del guion) y que compara con el valor calculado en el primer bucle (0x4BC):

```
00500AC1  .  33C9          xor     ecx, ecx
00500AC3  .  57            push    edi                   Arg1
00500AC4  .  66:890E       mov     word ptr [esi], cx
00500AC7  .  E8 14E00B00   call    005BEAE0              mirc.005BEAE0
00500ACC  .  83C4 04       add     esp, 0x4
00500ACF  .  3BE8          cmp     ebp, eax              // ** serial '1212-'
00500AD1  .^ 0F85 FFFEFFF  jnz     005009D6
00500AD7  .  BA 2D000000   mov     edx, 0x2D
00500ADC  .  66:8916       mov     word ptr [esi], dx
00500ADF  .  83C6 02       add     esi, 0x2
00500AE2  .  66:833E 00    cmp     word ptr [esi], 0x0
eax=000004BC   0x4BC== "1212"
ebp=0000199B   0x199B == "6555"
```

Lo que indica que nuestro serial deberá ser del tipo:

```
serial: 6555-xxxx
```

Si lo cambiamos a 6555-3434 y volvemos a probar vemos que estamos en lo cierto, pero tenemos otro problema en la siguiente comparación:

```
00500AE2  .   66:833E 00    cmp     word ptr [esi], 0x0
00500AE6  .^  0F84 EAFEFFF( je      005009D6
00500AEC  .   56            push    esi                      [Arg1
00500AED  .   E8 EEDF0B00   call    005BEAE0                  mirc.005BEAE0
00500AF2  .   83C4 04       add     esp, 0x4
00500AF5  .   33C9          xor     ecx, ecx
00500AF7  .   3BD8          cmp     ebx, eax                  // ** serial '-3434'
00500AF9  .   0F94C1        sete    cl
00500AFC  .   5F            pop     edi
00500AFD  .   5E            pop     esi
00500AFE  .   5D            pop     ebp
eax=00000D6A  <-- 0xD6A == "3434"
ebx=00090D47  <-- 0x90D47 == "593223"
```

Compara la segunda parte del serial (después del guion) con el valor calculado en el segundo loop. De esto se desprende que el serial debe ser de esta manera:

```
Nombre: Snat[ClS]
Serial: 6555-593223
```

Probamos estos datos y vemos que el retorno del CALL es EAX==0x1 y el JE posterior no se ejecuta:

```
0050281D  .   68 38D48200   push    0082D438                  UNICODE "6555-593223"
00502822  .   68 08F58200   push    0082F508                  UNICODE "Snat[ClS]"
00502827  .   E8 E4E2FFFF   call    00500B10                  // ** comprueba serial
0050282C  .   83C4 08       add     esp, 0x8
0050282F  .   85C0          test    eax, eax
00502831  .v  0F84 0D01000( je      00502944
00502837  .   68 68B38200   push    0082B368
0050283C  .   68 38D48200   push    0082D438                  UNICODE "6555-593223"
00502841  .   E8 DAE4FFFF   call    00500D20
00502846  .   A1 58B38200   mov     eax, dword ptr [0x82B358]
0050284B  .   83C4 08       add     esp, 0x8
```

Dentro del siguiente call (de 0x502841) continúa realizando operaciones:

```
00500D8F  .   33FF          xor     edi, edi
00500D91  >   8BC6          mov     eax, esi
00500D93  .   8D50 02       lea     edx, dword ptr [eax+0x2]
00500D96  >   66:8B08       mov     cx, word ptr [eax]
00500D99  .   83C0 02       add     eax, 0x2
00500D9C  .   66:85C9       test    cx, cx
00500D9F  .^  75 F5         jnz     short 00500D96
00500DA1  .   2BC2          sub     eax, edx
00500DA3  .   D1F8          sar     eax, 1
00500DA5  .   8D1446        lea     edx, dword ptr [esi+eax*2]
00500DA8  .   0FB6041F      movzx   eax, byte ptr [edi+ebx]
00500DAC  .   50            push    eax
00500DAD  .   8D4C24 10     lea     ecx, dword ptr [esp+0x10]
00500DB1  .   68 B47F7000   push    00707FB4                  UNICODE "%02x"
00500DB6  .   51            push    ecx
00500DB7  .   895424 18     mov     dword ptr [esp+0x18], edx
00500DBB  .   E8 F009F0FF   call    004017B0
00500DC0  .   47            inc     edi
00500DC1  .   83C4 0C       add     esp, 0xC
00500DC4  .   83FF 10       cmp     edi, 0x10
00500DC7  .^  7C C8         jl      short 00500D91
00500DC9  .   6A 02         push    0x2
00500DCB  .   8D9424 7C100( lea     edx, dword ptr [esp+0x107C]
```

Si miramos en el dump vemos que construyó una cadena:

```
Address   Hex dump                                          ASCII
0082B368  32 00 37 00 33 00 31 00 63 00 32 00 36 00 66 00  2.7.3.1.c.2.6.f.
0082B378  62 00 63 00 30 00 32 00 63 00 61 00 61 00 37 00  b.c.0.2.c.a.a.7.
0082B388  61 00 31 00 63 00 65 00 33 00 34 00 61 00 33 00  a.1.c.e.3.4.a.3.
0082B398  33 00 62 00 62 00 31 00 62 00 65 00 38 00 33 00  3.b.b.1.b.e.8.3.
0082B3A8  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082B3B8  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082B3C8  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082B3D8  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0082B3E8  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

De momento no sabemos para qué, seguimos traceando y vemos que continúa en un sinfín de loops donde va generando más y más valores que no nos dicen nada.

Cuando termina retorna del call:

```
00502800  .   83C4 18       add     esp, 0x18
00502803  .   66:391D 08F5  cmp     word ptr [0x82F508], bx
0050280A  .v  0F84 E001000  je      005029F0
00502810  .   66:391D 38D4  cmp     word ptr [0x82D438], bx
00502817  .v  0F84 D301000  je      005029F0
0050281D  .   68 38D48200   push    0082D438                 UNICODE "6555-593223"
00502822  .   68 08F58200   push    0082F508                 UNICODE "Snat[ClS]"
00502827  .   E8 E4E2FFFF   call    00500B10                 // ** comprueba serial
0050282C  .   83C4 08       add     esp, 0x8
0050282F  .   85C0          test    eax, eax
00502831  .v  0F84 0D01000  je      00502944
00502837  .   68 68B38200   push    0082B368                 UNICODE "2731C26fbC02CAA7A1cE34a33bb1be830B2694E5EDa6a986"
0050283C  .   68 38D48200   push    0082D438                 UNICODE "6555-593223"
00502841  .   E8 DAE4FFFF   call    00500D20
00502846  .   A1 58B38200   mov     eax, dword ptr [0x82B358]
0050284B  .   83C4 08       add     esp, 0x8
0050284E  .   3BC3          cmp     eax, ebx
00502850  .v  75 22         jnz     short 00502874
ds:[0082B358]=00000000
eax=00000036
```

Se ve el puntero a una de las strings que generó "2731C26…" y después la lectura de la variable 0x82B358 con valor igual a 0x0, el JNZ de 0x502850 no se ejecuta y llama a GetLastError con el parámetro ERROR_SUCCESS (parece que todo ha ido bien).

Continúa con unas operaciones que resultan en EAX==0xD y que salva en una variable global:

```
00502850  .v  75 22         jnz     short 00502874
00502852  .   E8 C5C90B00   call    005BF21C
00502857  .   99            cdq
00502858  .   B9 E8030000   mov     ecx, 0x3E8               // ** ECX == 1000d
0050285D  .   F7F9          idiv    ecx
0050285F  .   33C0          xor     eax, eax
00502861  .   81FA F401000  cmp     edx, 0x1F4               // ** cmp edx, 500d
00502867  .   0F9FC0        setg    al
0050286A  .   83C0 0C       add     eax, 0xC
0050286D  .   A3 58B38200   mov     dword ptr [0x82B358], eax
00502872  .v  EB 11         jmp     short 00502885
00502874  >   33C9          xor     ecx, ecx
eax=0000000D
ds:[0082B358]=00000000
```

Llegamos a otro call:

```
00502896  .   A1 58B38200   mov     eax, dword ptr [0x82B358]
0050289B  .   50            push    eax
0050289C  .   8D4C24 20     lea     ecx, dword ptr [esp+0x20]
005028A0  .   51            push    ecx
005028A1  .   E8 8A65F2FF   call    00428E30
005028A6  .   83C4 14       add     esp, 0x14
005028A9  .   3BC5          cmp     eax, ebp
005028AB  .v  0F85 9300000  jnz     00502944
005028B1  .   8D5424 10     lea     edx, dword ptr [esp+0x10]
005028B5  .   68 68B38200   push    0082B368                 UNICODE "2731C26fbC02CAA7A1cE
005028BA  .   52            push    edx
005028BB  .   E8 10EFEFFF   call    004017D0
```

Si entramos con <F7> vemos infinidad de strings como estas:

```
00428E30  r$  B8 F4220000   mov     eax, 0x22F4
00428E35  .   E8 16591900   call    005BE750
00428E3A  .   A1 888A7800   mov     eax, dword ptr [0x788A88]
00428E3F  .   33C4          xor     eax, esp
00428E41  .   898424 F0220  mov     dword ptr [esp+0x22F0], eax
00428E48  .   B8 9CFA6F00   mov     eax, 006FFA9C
00428E4D  .   53            push    ebx
00428E4E  .   8B9C24 FC220  mov     ebx, dword ptr [esp+0x22FC]
00428E55  .   56            push    esi
00428E56  .   57            push    edi
00428E57  .   33FF          xor     edi, edi
00428E59  .   C74424 0C 94  mov     dword ptr [esp+0xC], 007024  UNICODE "CacSziM="
00428E61  .   C74424 10 61  mov     dword ptr [esp+0x10], 0xEB6
00428E69  .   897C24 14     mov     dword ptr [esp+0x14], edi
00428E6D  .   894424 18     mov     dword ptr [esp+0x18], eax
00428E71  .   C74424 1C 58  mov     dword ptr [esp+0x1C], 00702  UNICODE "F+RIiiqaSn8jQV0iNgrLrPR7tw=="
00428E79  .   C74424 20 3D  mov     dword ptr [esp+0x20], 0xD9A
00428E81  .   C74424 24 01  mov     dword ptr [esp+0x24], 0x1
00428E89  .   894424 28     mov     dword ptr [esp+0x28], eax
00428E8D  .   C74424 2C 1C  mov     dword ptr [esp+0x2C], 00702  UNICODE "F+RIiiqaSn8jQV0iNgrLrPR79DDx"
00428E95  .   C74424 30 ED  mov     dword ptr [esp+0x30], 0xE61
00428E9D  .   C74424 34 02  mov     dword ptr [esp+0x34], 0x2
00428EA5  .   894424 38     mov     dword ptr [esp+0x38], eax
00428EA9  .   C74424 3C D0  mov     dword ptr [esp+0x3C], 00702  UNICODE "F+RIiiqaSn8jQV0iNgrLrPR7t2rxrobut3M="
00428EB1  .   C74424 40 69  mov     dword ptr [esp+0x40], 0xEEE
```

Lo que hace básicamente es buscar en una tabla el valor 0xD que tiene en EAX y que salvó en la variable 0x82B358, obteniendo una dirección de donde recupera otra cadena:

```
Address   Hex dump                                          ASCII
002107F0  46 00 2B 00 52 00 49 00 69 00 69 00 71 00 61 00  F.+.R.I.i.i.q.a.
00210800  53 00 6E 00 38 00 6A 00 51 00 56 00 30 00 69 00  S.n.8.j.Q.V.0.i.
00210810  4E 00 67 00 72 00 4C 00 72 00 50 00 52 00 37 00  N.g.r.L.r.P.R.7.
00210820  39 00 44 00 44 00 78 00 36 00 59 00 54 00 6C 00  9.D.D.x.6.Y.T.l.
00210830  59 00 51 00 76 00 59 00 43 00 4F 00 67 00 49 00  Y.Q.v.Y.C.O.g.I.
00210840  70 00 52 00 4E 00 6B 00 2B 00 2F 00 79 00 78 00  p.R.N.k.+./.y.x.
00210850  4B 00 50 00 62 00 4D 00 74 00 6E 00 58 00 6B 00  K.P.b.M.t.n.X.k.
00210860  73 00 35 00 65 00 4F 00 77 00 65 00 39 00 36 00  s.5.e.O.w.e.9.6.
00210870  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00210880  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00210890  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
002108A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
002108B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Pasamos este call:

```
00429387  .  50            push    eax
00429388  .  E8 23651700   call    0059F8B0
0042938D  .  8B0E          mov     ecx, dword ptr [esi]
0042938F  .  6A 02         push    0x2
00429391  .  51            push    ecx
00429392  .  8D9424 40020  lea     edx, dword ptr [esp+0x240]
00429399  .  68 D0200000   push    0x20D0
0042939E  .  52            push    edx
0042939F  .  E8 6C711700   call    005A0510
004293A4  .  6A 4C         push    0x4C
004293A6  .  6A FF         push    -0x1
004293A8  .  57            push    edi
004293A9  .  8D8424 54020  lea     eax, dword ptr [esp+0x254]
004293B0  .  68 98157000   push    00701598                 UNICODE "xyzzy"
004293B5  .  50            push    eax
004293B6  .  E8 758FFFFF   call    00422330
004293BB  .  8D8C24 5C020  lea     ecx, dword ptr [esp+0x25C]
004293C2  .  51            push    ecx
004293C3  .  E8 B8380400   call    0046CC80
```

Y volvemos a mirar la cadena:

```
Address   Hex dump                                          ASCII
002107F0  68 00 74 00 74 00 70 00 3A 00 2F 00 2F 00 77 00  h.t.t.p.:././.w.
00210800  77 00 77 00 2E 00 6D 00 69 00 72 00 63 00 2E 00  w.w...m.i.r.c...
00210810  63 00 6F 00 2E 00 75 00 6B 00 2F 00 63 00 67 00  c.o...u.k./.c.g.
00210820  69 00 2D 00 62 00 69 00 6E 00 2F 00 72 00 65 00  i.-.b.i.n./.r.e.
00210830  67 00 63 00 68 00 65 00 63 00 6B 00 2E 00 63 00  g.c.h.e.c.k...c.
00210840  67 00 69 00 3F 00 63 00 6F 00 64 00 65 00 3D 00  g.i.?.c.o.d.e.=.
00210850  00 00 50 00 62 00 4D 00 74 00 6E 00 58 00 6B 00  ..P.b.M.t.n.X.k.
00210860  73 00 35 00 65 00 4F 00 77 00 65 00 39 00 36 00  s.5.e.O.w.e.9.6.
00210870  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00210880  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00210890  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
002108A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
002108B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Mira por dónde, una validación on-line !

Cuando termina retorna de la función con EAX==0x1:

```
0050289B  .  50            push    eax
0050289C  .  8D4C24 20     lea     ecx, dword ptr [esp+0x20]
005028A0  .  51            push    ecx
005028A1  .  E8 8A65F2FF   call    00428E30                 // ** desencripta string
005028A6  .  83C4 14       add     esp, 0x14
005028A9  .  3BC5          cmp     eax, ebp
005028AB  .v 0F85 9300000  jnz     00502944
005028B1  .  8D5424 10     lea     edx, dword ptr [esp+0x10]
005028B5  .  68 68B38200   push    0082B368                 UNICODE "2731C26fbC02CAA7A1cE
005028BA  .  52            push    edx
005028BB  .  E8 10EFEFFF   call    004017D0
005028C0  .  A1 3C1E8300   mov     eax, dword ptr [0x831E3C]
005028C5  .  83C4 08       add     esp, 0x8
005028C8  .  3BC3          cmp     eax, ebx
005028CA  .v 74 09         je      short 005028D5
005028CC  .  50            push    eax
005028CD  .  E8 68CB0B00   call    005BF43A
005028D2  .  83C4 04       add     esp, 0x4
005028D5  >  68 54190000   push    0x1954
```

El JNZ no se produce y continúa con el siguiente call de 0x5028BB al que le pasa dos parámetros:

Un puntero a la string "2731C26.." y otro a la cadena que acaba de desencriptar:

```
ESP ==>  002128EC  UNICODE "http://www.mirc.co.uk/cgi-bin/regcheck.cgi?code="
ESP+4    0082B368  UNICODE "2731C26fbC02CAA7A1cE34a33bb1be830B2694E5EDa6a986"
ESP+8    00216AF4
ESP+C    00501D80  mirc.00501D80
ESP+10   00216AB8
ESP+14   00000000
ESP+18   00740068  mirc.00740068
ESP+1C   00700074  UNICODE "DISP_E_BADVARTYPE"
ESP+20   002F003A
```

Donde hace unas operaciones con las strings y enseguida retorna:

```
00502896  .   A1 58B38200   mov     eax, dword ptr [0x82B358]
0050289B  .   50            push    eax
0050289C  .   8D4C24 20     lea     ecx, dword ptr [esp+0x20]
005028A0  .   51            push    ecx
005028A1  .   E8 8A65F2FF   call    00428E30                    // ** desencripta string
005028A6  .   83C4 14       add     esp, 0x14
005028A9  .   3BC5          cmp     eax, ebp
005028AB  .v  0F85 9300000  jnz     00502944
005028B1  .   8D5424 10     lea     edx, dword ptr [esp+0x10]
005028B5  .   68 68B38200   push    0082B368                    UNICODE "2731C26fbC02CAA7A1cE34a33bt
005028BA  .   52            push    edx
005028BB  .   E8 10EFEFFF   call    004017D0
005028C0  .   A1 3C1E8300   mov     eax, dword ptr [0x831E3C]
005028C5  .   83C4 08       add     esp, 0x8
005028C8  .   3BC3          cmp     eax, ebx
005028CA  .v  74 09         je      short 005028D5
005028CC  .   50            push    eax
ds:[00831E3C]=00000000
eax=002128EC, (UNICODE "http://www.mirc.co.uk/cgi-bin/regcheck.cgi?code=2731C26fbC02CAA7A1cE34a33bb1be
```

Parece que concatenó las strings: "http://www.mirc.co.uk/.../?code=2731C26...", pasa por dos llamadas más sin consecuencias y después llega aquí:

```
005028FB  .   68 D8158300   push    008315D8                    UNICODE "http://www.mirc.co.uk/cgi-t
00502900  .   892D 441E830  mov     dword ptr [0x831E44], ebp
00502906  .   892D 282F830  mov     dword ptr [0x832F28], ebp
0050290C  .   E8 5FD5FBFF   call    004BFE70                    // ** Crea Hilo
00502911  .   83C4 20       add     esp, 0x20
00502914  .   53            push    ebx                         Timerproc
00502915  .   68 E8030000   push    0x3E8                       Timeout = 1000. ms
0050291A  .   6A 64         push    0x64                        TimerID = 64 (100.)
0050291C  .   891D 60B3820  mov     dword ptr [0x82B360], ebx
00502922  .   56            push    esi                         hWnd
00502923  .   891D 5CB3820  mov     dword ptr [0x82B35C], ebx
00502929  .   85C0          test    eax, eax
0050292B  .v  75 0A         jnz     short 00502937
0050292D  .   C705 5CB3820  mov     dword ptr [0x82B35C], 0x2
00502937  >   FF15 10F86F0  call    dword ptr [<&USER32.SetTime  SetTimer
0050293D  .   8BC5          mov     eax, ebp
0050293F  .v  E9 73010000   jmp     00502AB7
```

Entramos con <F7> y hay varios subcall, el el primero de ellos obtiene las direcciones de APIs tipo HttpOpenRequestW, HttpQueryInfoW, HttpSendRequestW, entre otras, todas ellas de la librería wininet.dll para terminar sacando un Thread:

```
005BFDE3  .   59            pop     ecx
005BFDE4  .   897E 54       mov     dword ptr [esi+0x54], edi
005BFDE7  .   3BC3          cmp     eax, ebx
005BFDE9  .v  75 03         jnz     short 005BFDEE
005BFDEB  .   8D45 10       lea     eax, [arg.3]
005BFDEE  >   50            push    eax                         pThreadId
005BFDEF  .   FF75 18       push    [arg.5]                     CreationFlags
005BFDF2  .   56            push    esi                         pThreadParm
005BFDF3  .   68 FAFC5B00   push    005BFCFA                    ThreadFunction = mirc.005BFCFA
005BFDF8  .   FF75 0C       push    [arg.2]                     StackSize
005BFDFB  .   FF75 08       push    [arg.1]                     pSecurity
005BFDFE  .   FF15 E4F26F0  call    dword ptr [<&KERNEL32.Creat  CreateThread
005BFE04  .   3BC3          cmp     eax, ebx
005BFE06  .v  75 20         jnz     short 005BFE28
005BFE08  .   FF15 54F36F0  call    dword ptr [<&KERNEL32.GetLa  GetLastError
005BFE0E  .   8945 FC       mov     [local.1], eax
005BFE11  >   56            push    esi
```

Interesante la dirección de inicio de éste 0x5BFCFA, si nos fijamos en la pila vemos que el parámetro CreationFlags es igual a 0x0 por lo que el hilo comenzará a ejecutar tan pronto le sea posible, pues no creándose suspendido (CreationFlags igual a 0x4) no es necesaria una posterior llamada a ResumeThread. Es evidente que la acción se desarrollará en el transcurso de ejecución de este hilo que de alguna manera deberá conectar a internet para validar el serial.

Si miramos la dirección de Inicio del Hilo vemos esto:

```
005BFCFA  .  8BFF          mov     edi, edi                          // ** Inicio Hilo
005BFCFC  .  55            push    ebp
005BFCFD  .  8BEC          mov     ebp, esp
005BFCFF  .  56            push    esi
005BFD00  .  E8 22A80000   call    005CA527
005BFD05  .  E8 17A80000   call    005CA521
005BFD0A  .  50            push    eax                               Arg1
005BFD0B  .  E8 F7A70000   call    005CA507                          mirc.005CA507
005BFD10  .  85C0          test    eax, eax
005BFD12  .v 75 2A         jnz     short 005BFD3E
005BFD14  .  8B75 08       mov     esi, dword ptr [ebp+0x8]
005BFD17  .  56            push    esi
005BFD18  .  E8 04A80000   call    005CA521
005BFD1D  .  50            push    eax                               Arg1
005BFD1E  .  E8 38A80000   call    005CA55B                          mirc.005CA55B
005BFD23  .  85C0          test    eax, eax
005BFD25  .v 75 0D         jnz     short 005BFD34
```

Colocamos un BP con <F2> como en la imagen, trataremos de investigarlo un poco, mientras tanto el hilo principal sigue su ejecución completando la rutina que gestiona la pulsación del botón <OK>:

```
00502911  .  83C4 20       add     esp, 0x20
00502914  .  53            push    ebx                               Timerproc
00502915  .  68 E8030000   push    0x3E8                             Timeout = 1000. ms
0050291A  .  6A 64         push    0x64                              TimerID = 64 (100.)
0050291C  .  891D 60B3820  mov     dword ptr [0x82B360], ebx
00502922  .  56            push    esi                               hWnd
00502923  .  891D 5CB3820  mov     dword ptr [0x82B35C], ebx
00502929  .  85C0          test    eax, eax
0050292B  .v 75 0A         jnz     short 00502937
0050292D  .  C705 5CB3820  mov     dword ptr [0x82B35C], 0x2
00502937  >  FF15 10F86F0  call    dword ptr [<&USER32.SetTime
0050293D  .  8BC5          mov     eax, ebp                          SetTimer
0050293F  .v E9 73010000   jmp     00502AB7                          // ** Salida
00502944  >  6A 02         push    0x2
00502946  .  68 38D48200   push    0082D438                          UNICODE "6555-593223"
0050294B  .  8D8C24 E8200  lea     ecx, dword ptr [esp+0x20E8]
00502952  .  68 D0200000   push    0x20D0
```

Ese JMP nos pone cerca del RET de la función que proseguirá su ejecución por User32.dll, no hace nada más, el resultado del registro depende de lo que haga el hilo, lo traceamos hasta llegar a este call:

```
005BFCCA  .  8365 FC 00    and     [local.1], 0x0
005BFCCE  .  FF70 58       push    dword ptr [eax+0x58]
005BFCD1  .  FF50 54       call    dword ptr [eax+0x54]              mirc.004BF750
005BFCD4  .  50            push    eax
005BFCD5  .  E8 A2FFFFFF   call    005BFC7C
005BFCDA  .  8B45 EC       mov     eax, [local.5]
005BFCDD  .  8B08          mov     ecx, dword ptr [eax]
005BFCDF  .  8B09          mov     ecx, dword ptr [ecx]
005BFCE1  .  894D E4       mov     [local.7], ecx
005BFCE4  .  50            push    eax
005BFCE5  .  51            push    ecx
005BFCE6  .  E8 F3B40000   call    005CB1DE
005BFCEB  .  59            pop     ecx
005BFCEC  .  59            pop     ecx
005BFCED  .  C3            ret
```

Es código del hilo, entramos con <F7> y vemos una función bastante grande:

```
004BF750  .  B8 78C30000   mov     eax, 0xC378
004BF755  .  E8 F6EF0F00   call    005BE750
004BF75A  .  A1 888A7800   mov     eax, dword ptr [0x788A88]
004BF75F  .  33C4          xor     eax, esp
004BF761  .  898424 74C30  mov     dword ptr [esp+0xC374], eax
004BF768  .  57            push    edi
004BF769  .  8BBC24 80C30  mov     edi, dword ptr [esp+0xC380]
004BF770  .  897C24 04     mov     dword ptr [esp+0x4], edi
004BF774  .  85FF          test    edi, edi
004BF776  .v 0F84 CF06000  je      004BFE4B
004BF77C  .  53            push    ebx
004BF77D  .  6A 02         push    0x2
004BF77F  .  57            push    edi
004BF780  .  8D8424 E4410  lea     eax, dword ptr [esp+0x41E4]
004BF787  .  68 D0200000   push    0x20D0
004BF78C  .  50            push    eax
004BF78D  .  E8 7E0D0E00   call    005A0510
004BF792  .  83C4 10       add     esp, 0x10
004BF795  .  BB 01000000   mov     ebx, 0x1
004BF79A  .  55            push    ebp
004BF79B  .  895C24 24     mov     dword ptr [esp+0x24], ebx
004BF79F  .  56            push    esi
004BF7A0  >  6A 02         push    0x2
004BF7A2  .  8D8C24 B8620  lea     ecx, dword ptr [esp+0x62B8]
004BF7A9  .  68 D0200000   push    0x20D0
004BF7AE  .  51            push    ecx
```

Termina en 0x4BFE62, traceamos con <F8> y vemos lo que hace:

```
004BF94F  .v 0F84 4C04000  je      004BFDA1
004BF955  .  6A 00         push    0x0
004BF957  .  6A 00         push    0x0
004BF959  .  6A 00         push    0x0
004BF95B  .  6A 00         push    0x0
004BF95D  .  68 DCFB6F00   push    006FFBDC                    UNICODE "mIRC"
004BF962  .  FFD0          call    eax                         WININET.InternetOpenW
004BF964  .  8BF0          mov     esi, eax
004BF966  .  897424 30     mov     dword ptr [esp+0x30], esi
004BF96A  .  85F6          test    esi, esi
004BF96C  .v 0F84 2F04000  je      004BFDA1
004BF972  .  6A 00         push    0x0
004BF974  .  6A 00         push    0x0
004BF976  .  6A 03         push    0x3
004BF978  .  6A 00         push    0x0
004BF97A  .  6A 00         push    0x0
004BF97C  .  6A 50         push    0x50
004BF97E  .  8D9424 CC620  lea     edx, dword ptr [esp+0x62CC]
004BF985  .  52            push    edx
004BF986  .  56            push    esi
004BF987  .  FF15 E83F810  call    dword ptr [0x813FE8]        WININET.InternetConnectW
004BF98D  .  8BE8          mov     ebp, eax
004BF98F  .  896C24 24     mov     dword ptr [esp+0x24], ebp
004BF993  .  85ED          test    ebp, ebp
004BF995  .v 0F84 FF03000  je      004BFD9A
004BF99B  .  66:837C24 44  cmp     word ptr [esp+0x44], 0x0
eax=7718AF85 (WININET.InternetOpenW)
```

Llama a InternetOpenW, InternetConnectW, etc.., hasta llegar a HttpSendRequestW donde falla (recordemos que estamos trabajando sin conexión a internet), por lo que este salto nos echa fuera:

```
004BFA46  .  6A 00         push    0x0
004BFA48  .  6A 00         push    0x0
004BFA4A  .  6A 00         push    0x0
004BFA4C  .  53            push    ebx
004BFA4D  .  FF15 E03F810  call    dword ptr [0x813FE0]        WININET.HttpSendRequestW
004BFA53  >  85C0          test    eax, eax
004BFA55  .v 0F84 2803000  je      004BFD83
004BFA5B  .  8B35 DC3F810  mov     esi, dword ptr [0x813FDC]   WININET.HttpQueryInfoW
004BFA61  .  33C0          xor     eax, eax
```

Llegamos a InternetCloseHandle, pero no desiste y continúa intentándolo pues esta vez llama a WS2_32.getaddrinfo con el parámetro www.mirc.co.uk, el retorno es EAX==2AF9, pasa a llamar a connect() con el socket '\Device\Afd' al que tampoco conecta, cierra el socket y retorna:

```
004BFDC7  .  E8 04EFFFFF   call    004BECD0                    // ** busca socket
004BFDCC  .  83C4 14       add     esp, 0x14
004BFDCF  .  83F8 02       cmp     eax, 0x2
004BFDD2  .v 75 4B         jnz     short 004BFE1F
004BFDD4  .  8B87 3C19000  mov     eax, dword ptr [edi+0x193C]
004BFDDA  .  85C0          test    eax, eax
004BFDDC  .v 74 13         je      short 004BFDF1
004BFDDE  .  50            push    eax
004BFDDF  .  E8 1CA2FDFF   call    0049A000
004BFDE4  .  83C4 04       add     esp, 0x4
004BFDE7  .  C787 3C19000  mov     dword ptr [edi+0x193C], 0x0
004BFDF1  >  8D87 D810000  lea     eax, dword ptr [edi+0x10D8]
004BFDF7  .  50            push    eax                         FileName
004BFDF8  .  FF15 FCF26F0  call    dword ptr [<&KERNEL32.Deletef  DeleteFileW
004BFDFE  >  8B4424 28     mov     eax, dword ptr [esp+0x28]
004BFE02  .  03C3          add     eax, ebx
004BFE04  .  83F8 0A       cmp     eax, 0xA
004BFE07  .  894424 28     mov     dword ptr [esp+0x28], eax
004BFE0B  .^ 0F8E 8FF9FFF  jle     004BF7A0
004BFE11  .v EB 2B         jmp     short 004BFE3E
004BFE13  >  C787 6C08000  mov     dword ptr [edi+0x86C], 0x2
004BFE1D  .v EB 29         jmp     short 004BFE48
004BFE1F  >  57            push    edi
004BFE20  .  E8 BBEDFFFF   call    004BEBE0
004BFE25  .  83C4 04       add     esp, 0x4
004BFE28  .  83BF 6C08000  cmp     dword ptr [edi+0x86C], 0x2
004BFE2F  .v 74 17         je      short 004BFE48
004BFE31  .  8D8F D810000  lea     ecx, dword ptr [edi+0x10D8]
004BFE37  .  51            push    ecx                         FileName
004BFE38  .  FF15 FCF26F0  call    dword ptr [<&KERNEL32.Deletef  DeleteFileW
004BFE3E  >  C787 6C08000  mov     dword ptr [edi+0x86C], 0x3
004BFE48  >  5E            pop     esi
004BFE49  .  5D            pop     ebp
004BFE4A  .  5B            pop     ebx
004BFE4B  >  8B8C24 78C30  mov     ecx, dword ptr [esp+0xC378]
004BFE52  .  5F            pop     edi
004BFE53  .  33CC          xor     ecx, esp
004BFE55  .  33C0          xor     eax, eax
004BFE57  .  E8 D8E80F00   call    005BE734
004BFE5C  .  81C4 78C3000  add     esp, 0xC378
004BFE62  .  C2 0400       ret     0x4
004BFE65     CC            int3
```

Termina llamando a DeleteFileW con el parámetro "" (no hay file) y modificando un flag:

```
004BFE1F  >  57              push    edi
004BFE20  .  E8 BBEDFFFF     call    004BEBE0
004BFE25  .  83C4 04         add     esp, 0x4
004BFE28  .  83BF 6C08000    cmp     dword ptr [edi+0x86C], 0x2
004BFE2F  .v 74 17           je      short 004BFE48
004BFE31  .  8D8F D810000    lea     ecx, dword ptr [edi+0x10D8]
004BFE37  .  51              push    ecx                          FileName
004BFE38  .  FF15 FCF26F0    call    dword ptr [<&KERNEL32.Deletef DeleteFileW
004BFE3E  >  C787 6C08000    mov     dword ptr [edi+0x86C], 0x3
004BFE48  >  5E              pop     esi
004BFE49  .  5D              pop     ebp
004BFE4A  .  5B              pop     ebx
ds:[00831E44]=00000001
Jumps from 004BF8C3, 004BF8D2, 004BF8EC, 004BFE11
```

Quizá esa sea la respuesta que espera el main thread, de aquí pasa a llamar a SetLastError:

```
005CA706  .  59              pop     ecx
005CA707  .  33F6            xor     esi, esi
005CA709  >  57              push    edi                          Error = ERROR_PATH_NOT_FOUND
005CA70A  .  FF15 00F26F0    call    dword ptr [<&KERNEL32.SetL.  SetLastError
005CA710  .  5F              pop     edi
005CA711  .  8BC6            mov     eax, esi
005CA713  .  5E              pop     esi
005CA714  .  C3              ret
```

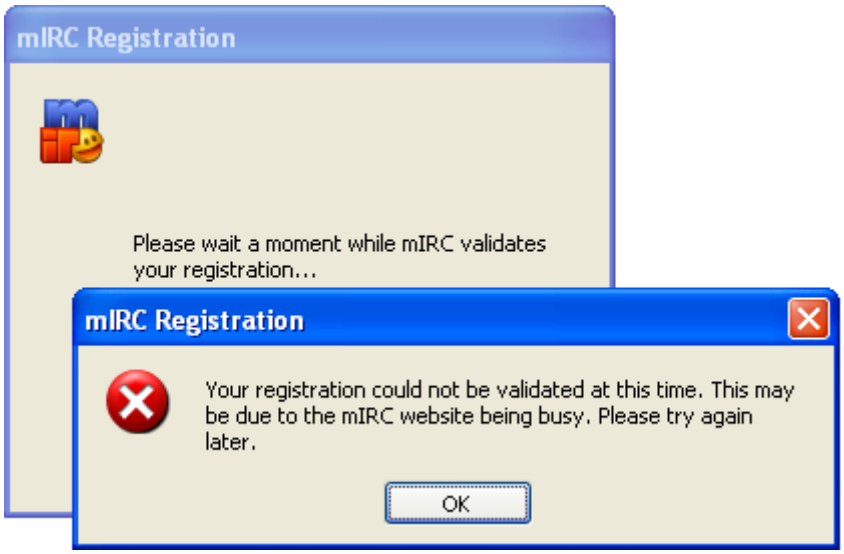
Estableciendo el error (archivo no encontrado), y el thread termina:

```
005BFCAE  .  59              pop     ecx
005BFCAF  >  FF75 08         push    [arg.1]                      ExitCode
005BFCB2  .  FF15 E8F26F0    call    dword ptr [<&KERNEL32.Exit   ExitThread
005BFCB8     CC              int3
005BFCB9  $  6A 0C           push    0xC
005BFCBB  .  68 D05E7700     push    00775ED0
005BFCC0  .  E8 2BB40000     call    005CB0F0
005BFCC5  .  E8 4BAA0000     call    005CA715
```

No hace nada más, pero de alguna forma debe informar al main thread si la validación fue o no correcta, colocamos un BPM en el flag que modificó el hilo 0x831E44 y damos RUN con <F9>, Olly para por lectura de este flag:

```
0050208F  >  A1 441E8300     mov     eax, dword ptr [0x831E44]    // ** lee respuesta hilo
00502094  .  3BC5            cmp     eax, ebp                     // ** ebp == 0x1
00502096  .v 0F84 190A000    je      00502AB5
0050209C  .  83F8 02         cmp     eax, 0x2
0050209F  .  A1 3C1E8300     mov     eax, dword ptr [0x831E3C]
005020A4  .v 74 15           je      short 005020BB
005020A6  .  3BC3            cmp     eax, ebx
005020A8  .v 74 15           je      short 005020BF
ds:[00831E44]=00000003
eax=00000001
```

Lo comparará con 0x1 (valor que tiene en EBP), el JE no se ejecuta y terminamos viendo este mensaje:



Hay que notar que ahora no dice aquello de "Your registration does not appear to be valid..", el mensaje ha cambiado a "Your registration could not be validated at this time..", que es muy diferente.

Si estudiamos un poco el proceso vemos que el main thread está loopeando a la espera de la respuesta de este hilo, esto lo vemos con las paradas que hace a causa del BPM en el flag de respuesta 0x831E44, traceando un poco el hilo principal se ve aquí:

```
00501F0C  .  A1 60B38200  mov   eax, dword ptr [0x82B360]   // ** Timeout
00501F11  .  03C5         add   eax, ebp
00501F13  .  83F8 3C      cmp   eax, 0x3C
00501F16  .  A3 60B38200  mov   dword ptr [0x82B360], eax   // ** INC
00501F1B  .v 7C 65        jl    short 00501F82
00501F1D  .  6A 64        push  0x64                        TimerID = 64 (100.)
00501F1F  .  56           push  esi                         hWnd
00501F20  .  FF15 18F86F0 call  dword ptr [<&USER32.KillTimer>] KillTimer
00501F26  .  33DB         xor   ebx, ebx
ds:[0082B360]=00000000
eax=00000000
```

El valor de 0x82B360 es 0x0, y se incrementa en 0x501F16 pasando a valer 0x1, sigue en el loop y verifica la respuesta del hilo:

```
0050208F  > A1 441E8300   mov   eax, dword ptr [0x831E44]   // ** lee respuesta hilo
00502094  .  3BC5         cmp   eax, ebp                    // ** ebp == 0x1
00502096  .v 0F84 190A000 je    00502AB5
0050209C  .  83F8 02      cmp   eax, 0x2
0050209F  .  A1 3C1E8300  mov   eax, dword ptr [0x831E3C]
005020A4  .v 74 15        je    short 005020BB
005020A6  .  3BC3         cmp   eax, ebx
005020A8  .v 74 15        je    short 005020BF
005020AA  .  50           push  eax
ds:[00831E44]=00000001
eax=00000001
```

Como todavía vale 0x1 significa que no completó su trabajo (recordemos que el thread puso este valor a 0x3 anteriormente, y hay otras zonas donde lo hace igual a 0x2, o igual al valor de EAX).

El hilo secundario continúa ejecutando código y el principal en este loop, vuelve a parar aquí:

```
00501F0C  .  A1 60B38200  mov   eax, dword ptr [0x82B360]   // ** Timeout
00501F11  .  03C5         add   eax, ebp
00501F13  .  83F8 3C      cmp   eax, 0x3C
00501F16  .  A3 60B38200  mov   dword ptr [0x82B360], eax   // ** INC
00501F1B  .v 7C 65        jl    short 00501F82
00501F1D  .  6A 64        push  0x64                        TimerID = 64 (100.)
00501F1F  .  56           push  esi                         hWnd
00501F20  .  FF15 18F86F0 call  dword ptr [<&USER32.KillTimer>] KillTimer
00501F26  .  33DB         xor   ebx, ebx
ds:[0082B360]=00000001
eax=00000000
```

El flag de espera aumentó en 1 unidad (de 0x0 a 0x1), mientras que su valor sea inferior a 0x3C (60d) no habrá problema y seguirá en el loop leyendo el flag de respuesta 0x831E44, y actualizando el de espera:

```
00501F0C  .  A1 60B38200  mov   eax, dword ptr [0x82B360]   // ** Timeout
00501F11  .  03C5         add   eax, ebp
00501F13  .  83F8 3C      cmp   eax, 0x3C
00501F16  .  A3 60B38200  mov   dword ptr [0x82B360], eax   // ** INC
00501F1B  .v 7C 65        jl    short 00501F82
00501F1D  .  6A 64        push  0x64                        TimerID = 64 (100.)
00501F1F  .  56           push  esi                         hWnd
00501F20  .  FF15 18F86F0 call  dword ptr [<&USER32.KillTimer>] KillTimer
00501F26  .  33DB         xor   ebx, ebx
ds:[0082B360]=00000002
eax=00000000
```

Si esperamos a que el hilo secundario finalice (dejando 0x831E44 igual a 0x3 en 0x4BFE3E) y modificamos voluntariamente este valor cambiándolo a 0xFFFFFFFF (-0x1) como en la siguiente imagen:

```
00502094  .  3BC5         cmp   eax, ebp                    // ** ebp == 0x1
00502096  .v 0F84 190A000 je    00502AB5
0050209C  .  83F8 02      cmp   eax, 0x2
0050209F  .  A1 3C1E8300  mov   eax, dword ptr [0x831E3C]
ebp=00000001
eax=FFFFFFFF
```

Ahora los valores no coinciden y el JE no se ejecuta, pasando a valorar 0x831E3C que está a 0x0.

Si miramos la salida del thread en IDA vemos que hay dos valores para el flag de respuesta:

```
004BFE11 jmp     short loc_4BFE3E

004BFE31 lea     ecx, [edi+10D8h]
004BFE37 push    ecx             ; lpFileName
004BFE38 call    ds:DeleteFileW

004BFE13
004BFE13 loc_4BFE13:
004BFE13 mov     dword ptr [edi+86Ch], 2
004BFE1D jmp     short loc_4BFE48

004BFE3E
004BFE3E loc_4BFE3E:
004BFE3E mov     dword ptr [edi+86Ch], 3

004BFE48
004BFE48 loc_4BFE48:
004BFE48 pop     esi
004BFE49 pop     ebp
004BFE4A pop     ebx

100.00% (-17,12158) (852,245) 000BF23E 004BFE3E: sub_4BF750:loc_4BFE3E (Synchronized with Hex View-1)
```

Siguiendo la rutina se comprueba que si flag==0x2 es que hubo respuesta, si flag==0x03 es que no la hubo. Además, en caso de haber respuesta ésta se almacena en una dirección de memoria y cuyo puntero se dejará en la dirección 0x831E3C, veamos la rutina del hilo principal (donde loopea):

```
0050208F  >  A1 441E8300   mov   eax, dword ptr [0x831E44]   // ** lee respuesta hilo
00502094  .  3BC5          cmp   eax, ebp                    // ** ebp == 0x1
00502096  .v 0F84 190A000( je    00502AB5
0050209C  .  83F8 02       cmp   eax, 0x2
0050209F  .  A1 3C1E8300   mov   eax, dword ptr [0x831E3C]   // ** puntero a reespuesta
005020A4  .v 74 15         je    short 005020BB              // ** BUENO
005020A6  .  3BC3          cmp   eax, ebx
005020A8  .v 74 15         je    short 005020BF
005020AA  .  50            push  eax
005020AB  .  E8 8AD30B00   call  005BF43A
005020B0  .  83C4 04       add   esp, 0x4
005020B3  .  891D 3C1E830( mov   dword ptr [0x831E3C], ebx
005020B9  .v EB 04         jmp   short 005020BF
005020BB  >  3BC3          cmp   eax, ebx
005020BD  .v 75 63         jnz   short 00502122                // ** BUENO
005020BF  >  6A 64         push  0x64                          TimerID = 64 (100.)
005020C1  .  56            push  esi                           hWnd
005020C2  .  FF15 18F86F0( call  dword ptr [<&USER32.KillTimer>]  KillTimer
005020C8  .  53            push  ebx
005020C9  .  53            push  ebx
005020CA  .  53            push  ebx
005020CB  .  53            push  ebx
005020CC  .  6A 04         push  0x4
005020CE  .  6A 03         push  0x3
005020D0  .  53            push  ebx
005020D1  .  53            push  ebx
005020D2  .  68 8A070000   push  0x78A                         // ** license couln't be validated
005020D7  .  E8 3477F2FF   call  00429810
005020DC  .  83C4 08       add   esp, 0x8
[00831E3C]=00000000
```

Este es el punto clave de la operación, una vez el hilo secundario es lanzado el principal queda en este loop a la espera de que 0x831E44 != 0x1 (el hilo no terminó su labor), cuando lo haga entonces será:

- ✗ `0x831E44 == 0x3 : No hubo contacto con el server.`
- ✓ `0x831E44 == 0x2 : Si hubo contacto.`
  - `0x831E3C : Puntero a respuesta del server.`

Pero la respuesta no puede ser cualquiera pues la revisa.

Comenzaremos por obligar al thread secundario a retornar un STATUS==OK cambiando:

```
004BFE94  .   E8 E4FE0F00   call    005BFD7D                          // ** lanza Thread
004BFE99  .   83C4 18       add     esp, 0x18
004BFE9C  .   85C0          test    eax, eax
004BFE9E  .v  75 0D         jnz     short 004BFEAD
004BFEA0  .   C786 6C08000  mov     dword ptr [esi+0x86C], 0x3        // ** STATUS_ERROR
004BFEAA  .   5E            pop     esi
004BFEAB  .   59            pop     ecx
004BFEAC  .   C3            ret
004BFEAD  >   8986 4019000  mov     dword ptr [esi+0x1940], eax
004BFEB3  .   B8 01000000   mov     eax, 0x1
004BFEB8  .   5E            pop     esi
004BFEB9  .   59            pop     ecx
004BFEBA  .   C3            ret
```

Por esto:

```
004BFE94  .   E8 E4FE0F00   call    005BFD7D                          // ** lanza Thread
004BFE99  .   83C4 18       add     esp, 0x18
004BFE9C  .   85C0          test    eax, eax
004BFE9E  .v  75 0D         jnz     short 004BFEAD
004BFEA0      C786 6C08000  mov     dword ptr [esi+0x86C], 0x2        // ** STATUS_OK
004BFEAA  .   5E            pop     esi
004BFEAB  .   59            pop     ecx
004BFEAC  .   C3            ret
004BFEAD  >   8986 4019000  mov     dword ptr [esi+0x1940], eax
004BFEB3  .   B8 01000000   mov     eax, 0x1
004BFEB8  .   5E            pop     esi
004BFEB9  .   59            pop     ecx
004BFEBA  .   C3            ret
```

haciendo esta modificación retornamos siempre un STATUS correcto (en caso de fallar la creación del thread), en caso contrario el mismo hilo deberá reportar este STATUS, cambiamos:

```
004BFE25  .   83C4 04       add     esp, 0x4
004BFE28  .   83BF 6C08000  cmp     dword ptr [edi+0x86C], 0x2
004BFE2F  .v  74 17         je      short 004BFE48
004BFE31  .   8D8F D810000  lea     ecx, dword ptr [edi+0x10D8]
004BFE37  .   51            push    ecx                               FileName
004BFE38  .   FF15 FCF26F0  call    dword ptr [<&KERNEL32.DeleteFile  DeleteFileW
004BFE3E  >   C787 6C08000  mov     dword ptr [edi+0x86C], 0x3
004BFE48  >   5E            pop     esi
004BFE49  .   5D            pop     ebp
004BFE4A  .   5B            pop     ebx
004BFE4B  >   8B8C24 78C30  mov     ecx, dword ptr [esp+0xC378]
004BFE52  .   5F            pop     edi
004BFE53  .   33CC          xor     ecx, esp
```

Por esto:

```
004BFE25  .   83C4 04       add     esp, 0x4
004BFE28  .   83BF 6C08000  cmp     dword ptr [edi+0x86C], 0x2
004BFE2F  .v  74 17         je      short 004BFE48
004BFE31  .   8D8F D810000  lea     ecx, dword ptr [edi+0x10D8]
004BFE37  .   51            push    ecx                               FileName
004BFE38  .   FF15 FCF26F0  call    dword ptr [<&KERNEL32.DeleteFile  DeleteFileW
004BFE3E      C787 6C08000  mov     dword ptr [edi+0x86C], 0x2        // ** STATUS_OK
004BFE48  >   5E            pop     esi
004BFE49  .   5D            pop     ebp
004BFE4A  .   5B            pop     ebx
004BFE4B  >   8B8C24 78C30  mov     ecx, dword ptr [esp+0xC378]
004BFE52  .   5F            pop     edi
004BFE53  .   33CC          xor     ecx, esp
```

De esta manera reportará un estado de OK pase lo que pase.

Vamos a por la respuesta, asignamos un puntero en la dirección 0x831E3C como en la imagen:

```
0050208F  >   A1 441E8300   mov     eax, dword ptr [0x831E44]         // ** lee respuesta hilo
00502094  .   3BC5          cmp     eax, ebp                          // ** ebp == 0x1
00502096  .v  0F84 190A000  je      00502AB5
0050209C  .   83F8 02       cmp     eax, 0x2
0050209F  .   A1 3C1E8300   mov     eax, dword ptr [0x831E3C]         // ** puntero a reespuesta
005020A4  .v  74 15         je      short 005020BB                    // ** BUENO
005020A6  .   3BC3          cmp     eax, ebx
005020A8  .v  74 15         je      short 005020BF
005020AA  .   50            push    eax
005020AB  .   E8 8AD30B00   call    005BF43A
005020B0  .   83C4 04       add     esp, 0x4
005020B3  .   891D 3C1E830  mov     dword ptr [0x831E3C], ebx
005020B9  .v  EB 04         jmp     short 005020BF
ds:[00831E3C]=00831E4C
eax=00000002
```

Asignamos un puntero

Y en la dirección apuntada 0x831E4C iremos escribiendo lo que veamos que busca:

```
00502149  .  6A 02          push    0x2
0050214B  .  8D4C24 14      lea     ecx, dword ptr [esp+0x14]
0050214F  .  68 ACC57000    push    0070C5AC                          UNICODE "redirect="
00502154  .  51             push    ecx
00502155  .  E8 16D80900    call    0059F970                          // ** busca 'redirect=XXXX'
0050215A  .  83C4 0C        add     esp, 0xC
0050215D  .  85C0           test    eax, eax
0050215F  .v 0F84 5009000   je      00502AB5
00502165  >  6A 64          push    0x64                              TimerID = 64 (100.)
00502167  .  56             push    esi                               hWnd
00502168  .  FF15 18F86F0   call    dword ptr [<&USER32.KillTimer>]   KillTimer
0050216E  .  8D5424 10      lea     edx, dword ptr [esp+0x10]
00502172  .  68 DC017000    push    007001DC                          UNICODE "♪◙"
00502177  .  52             push    edx
00502178  .  E8 A35B0700    call    00577D20
0050217D  .  6A 02          push    0x2
0050217F  .  8D4424 1C      lea     eax, dword ptr [esp+0x1C]
00502183  .  68 9CC57000    push    0070C59C                          UNICODE "id=mirc"
00502188  .  50             push    eax
00502189  .  E8 E2D70900    call    0059F970                          // ** busca 'id=mirc'
0050218E  .  83C4 14        add     esp, 0x14
00502191  .  3BC3           cmp     eax, ebx
00502193  .v 0F84 4705000   je      <CANT_VALIDATED>
00502199  .  6A 02          push    0x2
0050219B  .  68 80C57000    push    0070C580                          UNICODE "status=ready"
005021A0  .  50             push    eax
005021A1  .  E8 CAD70900    call    0059F970
005021A6  .  8BF8           mov     edi, eax
005021A8  .  83C4 0C        add     esp, 0xC
005021AB  .  3BFB           cmp     edi, ebx
005021AD  .v 0F84 2D05000   je      <CANT_VALIDATED>
005021B3  .  6A 02          push    0x2
005021B5  .  68 ACC57000    push    0070C5AC                          UNICODE "redirect="
005021BA  .  57             push    edi
005021BB  .  E8 B0D70900    call    0059F970
005021C0  .  83C4 0C        add     esp, 0xC
005021C3  .  3BC3           cmp     eax, ebx
005021C5  .v 0F84 6C01000   je      00502337                          // ** NO_REVALIDA
005021CB  .  8D78 12        lea     edi, dword ptr [eax+0x12]
```

Necesita los valores:

- `redirect=ALGO`
- `id=mirc`
- `status=ready`

Además deberá saltar en 0x5021C5 para que no vuelva a llamar al Thread para contactar de nuevo con el servidor. Lo vamos anotando, y seguimos mirando:

```
00502337  >  6A 02          push    0x2
00502339  .  68 58C57000    push    0070C558                          UNICODE "valid="
0050233E  .  57             push    edi
0050233F  .  E8 2CD60900    call    0059F970
00502344  .  8BF8           mov     edi, eax
00502346  .  83C4 0C        add     esp, 0xC
00502349  .  3BFB           cmp     edi, ebx
0050234B  .v 0F84 8F03000   je      <CANT_VALIDATED>
00502351  .  83C7 0C        add     edi, 0xC
00502354  .  66:391F        cmp     word ptr [edi], bx
00502357  .v 0F84 8303000   je      <CANT_VALIDATED>
0050235D  .  57             push    edi                               Arg1
0050235E  .  E8 93C70B00    call    005BEAF6                          mirc.005BEAF6
00502363  .  83C4 04        add     esp, 0x4
00502366  .  3BC5           cmp     eax, ebp
00502368  .v 0F85 9A01000   jnz     <INVALID_CODE>
0050236E  .  6A 02          push    0x2
00502370  .  68 4CC57000    push    0070C54C                          UNICODE "sig="
00502375  .  57             push    edi
```

Seguimos apuntando:

- `valid=ALGO`
- `sig=ALGO_MAS`

Además NO debe de saltar en 0x502368 donde está el mensaje de Invalid Key que vimos antes.

Y nos quedará otro obstáculo.

Es este:

```
00502411  .  55              push    ebp
00502412  .  68 68B38200     push    0082B368          UNICODE "2731c26fbc02CAA7A1cE34a33bB1Be830b2
00502417  .  68 38D48200     push    0082D438          UNICODE "6555-593223"
0050241C  .  E8 9FEBFFFF     call    00500FC0
00502421  .  83C4 14         add     esp, 0x14
00502424  .  85C0            test    eax, eax
00502426  .v 74 1E           je      short 00502446    // ** MALO
00502428  .  55              push    ebp
00502429  .  68 68B38200     push    0082B368          UNICODE "2731c26fbc02CAA7A1cE34a33bB1Be830b2
0050242E  .  68 38D48200     push    0082D438          UNICODE "6555-593223"
00502433  .  68 08F58200     push    0082F508          UNICODE "Snat[ClS]"
00502438  .  56              push    esi
00502439  .  E8 12F6FFFF     call    <CHECK>
0050243E  .  83C4 14         add     esp, 0x14
00502441  .v E9 6F060000     jmp     00502AB5
00502446  >  53              push    ebx
00502447  .  53              push    ebx
00502448  .  53              push    ebx
00502449  .  53              push    ebx
```

Anotamos que tampoco saltará en 0x502426 ya que en el call CHECK se escriben unas valiosas claves en el registro de sistema:

```
00501BB3  .  68 300F8200     push    00820F30
00501BB8  .  E8 C3D70B00     call    005BF380
00501BBD  .  6A 14           push    0x14
00501BBF  .  56              push    esi
00501BC0  .  68 08C57000     push    0070C508          UNICODE "UserName"
00501BC5  .  68 01000080     push    0x80000001
00501BCA  .  E8 E1E4FFFF     call    005000B0
00501BCF  .  6A 14           push    0x14
00501BD1  .  57              push    edi
00501BD2  .  68 F8C47000     push    0070C4F8          UNICODE "License"
00501BD7  .  68 01000080     push    0x80000001
00501BDC  .  E8 CFE4FFFF     call    005000B0
00501BE1  .  8B5424 64       mov     edx, dword ptr [es
00501BE5  .  83C4 4C         add     esp, 0x4C
00501BE8  .  6A 54           push    0x54
00501BEA  .  52              push    edx
00501BEB  .  68 E4C47000     push    0070C4E4          UNICODE "Validated"
00501BF0  .  68 01000080     push    0x80000001
00501BF5  .  E8 B6E4FFFF     call    005000B0
```

Esas son. Y ahora que sabemos esto es momento de dar un golpe de efecto, como no nos gusta que el thread ande contactando con ningún server aprovecharemos la rutina interna que utiliza para los trabajos, con inicio en 0x4BF750 y que intenta acceder a Internet para injertar algo de código SIMULANDO una respuesta del server xDD

La cadena de respuesta podría ser algo así:

- ✓ id=mircstatus=readyvalid=yessig=121212redirect=7878

Como no sabemos con certeza lo que debemos esperar empezaremos con esta.

Modificamos la rutina del thread de esta manera:

```
004BF750     60              pushad
004BF751     C705 3C1E830    mov     dword ptr [0x831E3C], 004BF7A0    // ** puntero respuesta
004BF75B     C705 441E830    mov     dword ptr [0x831E44], 0x2         // ** STATUS_OK
004BF765     61              popad
004BF766     33C0            xor     eax, eax
004BF768     C2 0400         ret     0x4
004BF76B     90              nop
004BF76C     90              nop
004BF76D     90              nop
004BF76E     90              nop
004BF76F     90              nop
004BF770  .  897C24 04       mov     dword ptr [esp+0x4], edi
004BF774  .  85FF            test    edi, edi
004BF776  .v 0F84 CF06000    je      004BFE4B
004BF77C  .  53              push    ebx
004BF77D  .  6A 02           push    0x2
004BF77F  .  57              push    edi
004BF780  .  8D8424 E4410    lea     eax, dword ptr [esp+0x41E4]
004BF787  .  68 D0200000     push    0x20D0
```

Lo que hace básicamente es establecer el puntero de respuesta a la zona 0x4BF7A0 donde ubicaremos la string que acabamos de armar.

Por otra parte setea el flag de STATUS a OK (0x831E44), hace EAX==0x0 (como hace el thread) y retorna con un RET 0x4, importante porque debe sacar un argumento de la pila.

copiamos la string al búfer:

```
Address   Hex dump                                          ASCII
004BF7A0  69 64 3D 6D 69 72 63 73 74 61 74 75 73 3D 72 65  id=mircstatus=re
004BF7B0  61 64 79 76 61 6C 69 64 3D 79 65 73 73 69 67 3D  adyvalid=yessig=
004BF7C0  31 32 31 32 31 32 72 65 64 69 72 65 63 74 3D 37  121212redirect=7
004BF7D0  38 37 38 00 00 00 00 00 00 00 00 00 00 00 00 00  878.............
004BF7E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004BF7F0  20 00 00 51 E8 17 0D 0E 00 8D 94 24 C4 62 00 00   ..Q♦♣.♫.ïö$─b..
004BF800  6A 2F 52 E8 26 F0 0F 00 8B F0 83 C4 18 85 F6 75  j/R♦&-☼.ï-ä─↑à÷u
004BF810  18 8D 84 24 B4 62 00 00 6A 3F 50 E8 0E F0 0F 00  ↑ïä$┤b..j?P♦♫-☼.
004BF820  8B F0 83 C4 08 85 F6 74 1D 6A 02 56 8D 8C 24 1C  ï-ä─◘à÷t↔j☻Vìï$∟
004BF830  21 00 00 68 D0 20 00 00 51 E8 D2 0C 0E 00 83 C4  !..h╨ ..Q♦Ò.♫.ä─
```

Estamos utilizando espacio de la rutina del thread que ya no se utilizará más.

Hemos completado una parte, modifiquemos ahora el main thread:

```
005021B3  .   6A 02        push    0x2
005021B5  .   68 ACC57000  push    0070C5AC                 UNICODE "redirect="
005021BA  .   57           push    edi
005021BB  .   E8 B0D70900  call    0059F970
005021C0  .   83C4 0C      add     esp, 0xC
005021C3  .   3BC3         cmp     eax, ebx
005021C5  ~   E9 6D010000  jmp     00502337                 // ** NO_REVALIDA
005021CA      90           nop
005021CB  .   8D78 12      lea     edi, dword ptr [eax+0x12]
005021CE  .   0FB707       movzx   eax, word ptr [edi]
005021D1  .   66:3BC3      cmp     ax, bx
005021D4  .v  0F84 0301000 je      005022DD
005021DA  .   66:83F8 22   cmp     ax, 0x22
```

Esto hace que no revalide los datos, luego este:

```
00502368      90           nop
00502369      90           nop
0050236A      90           nop
0050236B      90           nop
0050236C      90           nop
0050236D      90           nop
0050236E  .   6A 02        push    0x2
00502370  .   68 4CC57000  push    0070C54C                 UNICODE "sig="
00502375  .   57           push    edi
00502376  .   E8 F5D50900  call    0059F970
0050237B  .   8BF8         mov     edi, eax
0050237D  .   83C4 0C      add     esp, 0xC
00502380  .   3BFB         cmp     edi, ebx
```

Para que no vaya al Invalid Key, más este otro:

```
0050241C  .   E8 9FEBFFFF  call    00500FC0
00502421  .   83C4 14      add     esp, 0x14
00502424  .   85C0         test    eax, eax
00502426      90           nop                              // ** MALO
00502427      90           nop
00502428  .   55           push    ebp
00502429  .   68 68B38200  push    0082B368                 UNICODE "2731c26fbc02CAA7A
0050242E  .   68 38D48200  push    0082D438                 UNICODE "6555-593223"
00502433  .   68 08F58200  push    0082F508                 UNICODE "Snat[ClS]"
00502438  .   56           push    esi
00502439  .   E8 12F6FFFF  call    <CHECK>
0050243E  .   83C4 14      add     esp, 0x14
00502441  .v  E9 6F060000  jmp     00502AB5
```

Para que entre en la rutina CHECK donde escribirá las claves de registro que enseguida veremos.

Ponemos dos breakpoints para verificar la corrección del método, el primero en el inicio de la rutina modificada 0x4BF750 y el segundo en la lectura del flag de respuesta 0x50209F, colocamos los datos en la ventana de registro que ya conocemos:

```
Nombre: Snat[ClS]
Serial: 6555-593223
```

Y aceptamos en la ventana de registro pulsando <OK>, Olly para en la rutina del thread:

```
004BF750    60            pushad
004BF751    C705 3C1E830  mov     dword ptr [0x831E3C], 004BF7A0     // ** puntero respuesta
004BF75B    C705 441E830  mov     dword ptr [0x831E44], 0x2          // ** STATUS_OK
004BF765    61            popad
004BF766    33C0          xor     eax, eax
004BF768    C2 0400       ret     0x4
004BF76B    90            nop
004BF76C    90            nop
004BF76D    90            nop
004BF76E    90            nop
004BF7A0=004BF7A0
ds:[00831E3C]=004BF7A0 (mirc.004BF7A0)
```

Establece el puntero, fija el STATUS a OK y retorna bien, pulsamos <F9> y caemos en el hilo principal, vemos que hasta ahora todo está funcionando bien:

```
0050208F  >  A1 441E8300  mov     eax, dword ptr [0x831E44]          // ** lee respuesta hilo
00502094  .  3BC5         cmp     eax, ebp                           // ** ebp == 0x1
00502096  .v 0F84 190A000 je      00502AB5
0050209C  .  83F8 02      cmp     eax, 0x2
0050209F  .  A1 3C1E8300  mov     eax, dword ptr [0x831E3C]          // ** puntero a reespuesta
005020A4  .v 74 15        je      short 005020BB                     // ** BUENO
005020A6  .  3BC3         cmp     eax, ebx
005020A8  .v 74 15        je      short 005020BF
005020AA  .  50           push    eax
005020AB  .  E8 8AD30B00  call    005BF43A
ds:[00831E3C]=004BF7A0
eax=00000002
```

Lee el puntero donde está nuestra string:

```
Address   Hex dump                                         ASCII
004BF7A0  69 64 3D 6D 69 72 63 73 74 61 74 75 73 3D 72 65  id=mircstatus=re
004BF7B0  61 64 79 76 61 6C 69 64 3D 79 65 73 73 69 67 3D  adyvalid=yessig=
004BF7C0  31 32 31 32 31 32 72 65 64 69 72 65 63 74 3D 37  121212redirect=7
004BF7D0  38 37 38 00 00 00 00 00 00 00 00 00 00 00 00 00  878.............
004BF7E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004BF7F0  20 00 00 51 E8 17 0D 0E 00 8D 94 24 C4 62 00 00   ..Q♦‡.♫.ïö$─b..
004BF800  6A 2F 52 E8 26 F0 0F 00 8B F0 83 C4 18 85 F6 75  j/R♦&-☼.ï-ä─↑à÷u
004BF810  18 8D 84 24 B4 62 00 00 6A 3F 50 E8 0E F0 0F 00  ↑ïä$┤b..j?P♦♫-☼.
004BF820  8B F0 83 C4 08 85 F6 74 1D 6A 02 56 8D 8C 24 1C  ï-ä─◘à÷t↔j☻Vìï$∟
004BF830  21 00 00 68 D0 20 00 00 51 E8 D2 0C 0E 00 83 C4  !..h╨ ..Q♦È.♫.ä─
```

Traceamos la rutina con <F8> y verificamos el recorrido seguido:

```
0050218E  .  83C4 14      add     esp, 0x14
00502191  .  3BC3         cmp     eax, ebx
00502193  .v 0F84 4705000 je      <CANT_VALIDATED>
00502199  .  6A 02        push    0x2
0050219B  .  68 80C57000  push    0070C580                           UNICODE "status=ready"
005021A0  .  50           push    eax
005021A1  .  E8 CAD70900  call    0059F970
005021A6  .  8BF8         mov     edi, eax
005021A8  .  83C4 0C      add     esp, 0xC
005021AB  .  3BFB         cmp     edi, ebx
005021AD  .v 0F84 2D005000 je     <CANT_VALIDATED>
005021B3  .  6A 02        push    0x2
005021B5  .  68 ACC57000  push    0070C5AC                           UNICODE "redirect="
005021BA  .  57           push    edi
005021BB  .  E8 B0D70900  call    0059F970
005021C0  .  83C4 0C      add     esp, 0xC
005021C3  .  3BC3         cmp     eax, ebx
005021C5  v  E9 6D010000  jmp     00502337                           // ** NO_REVALIDA
005021CA     90           nop
```

Por ahora vamos bien, hemos sorteado los saltos al mensaje "Your registration could not be validated at this time..", llegamos a los otros dos saltos MALOS que tampoco se producen:

```
00502337  >  6A 02        push    0x2
00502339  .  68 58C57000  push    0070C558                           UNICODE "valid="
0050233E  .  57           push    edi
0050233F  .  E8 2CD60900  call    0059F970
00502344  .  8BF8         mov     edi, eax
00502346  .  83C4 0C      add     esp, 0xC
00502349  .  3BFB         cmp     edi, ebx
0050234B  .v 0F84 8F03000 je      <CANT_VALIDATED>
00502351  .  83C7 0C      add     edi, 0xC
00502354  .  66:391F      cmp     word ptr [edi], bx
00502357  .v 0F84 8303000 je      <CANT_VALIDATED>
0050235D  .  57           push    edi                                Arg1
0050235E  .  E8 93C70B00  call    005BEAF6                           mirc.005BEAF6
```

Y conseguimos llegar al call CHECK como queríamos:

```
00502421  .  83C4 14      add     esp, 0x14
00502424  .  85C0         test    eax, eax
00502426     90           nop                                  // ** MALO
00502427     90           nop
00502428  .  55           push    ebp
00502429  .  68 68B38200  push    0082B368                     UNICODE "2731C26fbC02cAA7A1Ce3
0050242E  .  68 38D48200  push    0082D438                     UNICODE "6555-593223"
00502433  .  68 08F58200  push    0082F508                     UNICODE "Snat[ClS]"
00502438  .  56           push    esi
00502439  .  E8 12F6FFFF  call    <CHECK>
0050243E  .  83C4 14      add     esp, 0x14
00502441  .v E9 6F060000  jmp     00502AB5
00502446  >  53           push    ebx
```

En el interior se crean las claves de registro a partir de aquí:

```
00501B62  .  83F8 01      cmp     eax, 0x1
00501B65  .v 0F85 3501000 jnz     00501CA0
00501B6B  .  8B7424 0C    mov     esi, dword ptr [esp+0xC]     mirc.0082F508; Case 1 of switc
00501B6F  .  57           push    edi                          // ** CORRECTO
00501B70  .  6A 02        push    0x2
00501B72  .  56           push    esi
00501B73  .  68 D0200000  push    0x20D0
00501B78  .  68 90CD8100  push    0081CD90
00501B7D  .  C705 44B3820 mov     dword ptr [0x82B344], 0x0
00501B87  .  C705 8CCD810 mov     dword ptr [0x81CD8C], 0x0
00501B91  .  E8 7AE90900  call    005A0510
00501B96  .  8B7C24 24    mov     edi, dword ptr [esp+0x24]
00501B9A  .  6A 02        push    0x2
```

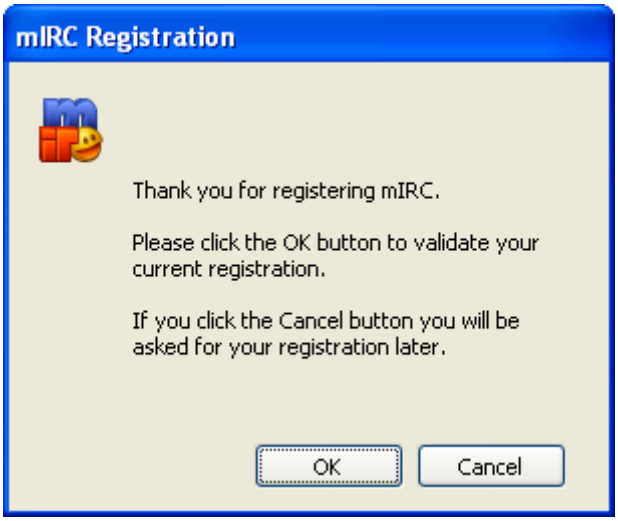
Si dejamos correr nos saca el mensaje de felicitación:

Nos muestra el cartel de felicitación. Lo aceptamos y entramos al programa, como observación veremos que en el menú Help ha desaparecido la opción Register.

Reiniciamos Olly y reactivamos las modificaciones desde la ventana View -> Patches:

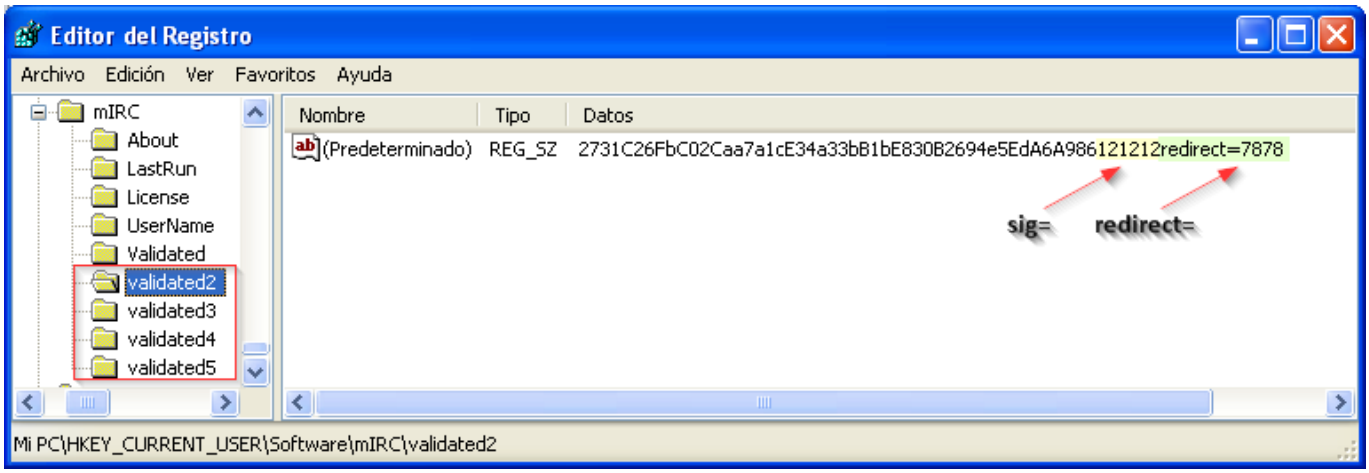
Address	Size	State	Old	New	Comment
004BF750	32.	Active	mov eax, 0xC378	pushad	
004BF7A0	80.	Active	push 0x2	imul esp, dword ptr [ebp+edi+0x6D], 0x73637269	
004BFE3E	10.	Removed	mov dword ptr [edi+0x86C], (	mov dword ptr [edi+0x86C], 0x2	// ** STATUS_OK
004BFEA0	10.	Removed	mov dword ptr [esi+0x86C], (	mov dword ptr [esi+0x86C], 0x2	// ** STATUS_OK
005021C5	6.	Active	je 00502337	jmp 00502337	// ** NO_REVALIDA
00502368	6.	Active	jnz <INVALID_CODE>	nop	
00502426	2.	Active	je short 00502446	nop	// ** MALO

Los dejamos en Active pulsando la barra espaciadora sobre todos ellos a excepción de las dos direcciones donde se modifica el flag de STATUS (0x4BFE3E y 0x4BFEA0) que dejamos sin modificar (no nos hacen falta), damos RUN y el programa arranca pero nos muestra este cuadro de diálogo:



Si pulsamos <OK> nos muestra el mensaje de felicitación otra vez y también la nag inicial (aún configurada para que no se muestre), después el programa arranca con normalidad.

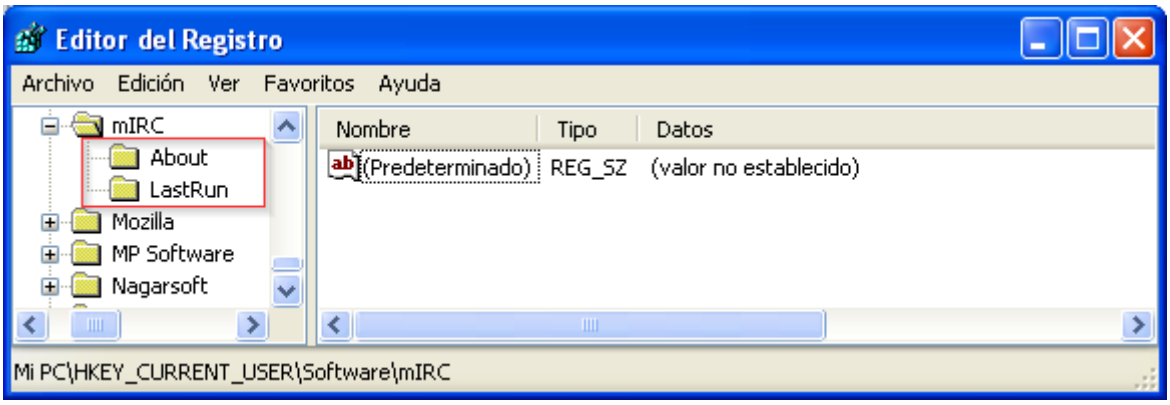
Si miramos en el registro veremos que ahora hay más claves:



Han aparecido validated2, validated3, validated4 y validated5 que antes no estaban, su contenido es la string larga que vimos antes "2731C26.." concatenada con el contenido de sig= de la falsa respuesta del server más el campo redirect al completo.

Tienen este mismo valor las claves validated3, validated4 y validated5.

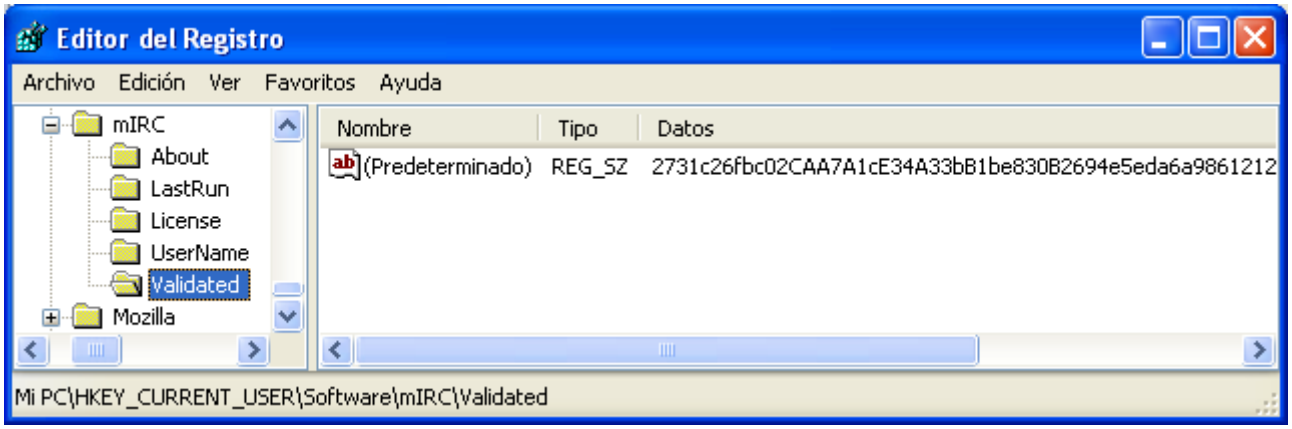
Si en lugar de pulsar <OK> en la ventana de validación pulsamos <Cancel> el programa arranca directamente sin mostrar el About pero borra las claves del registro:



Y vuelve a estar en modo Trial.

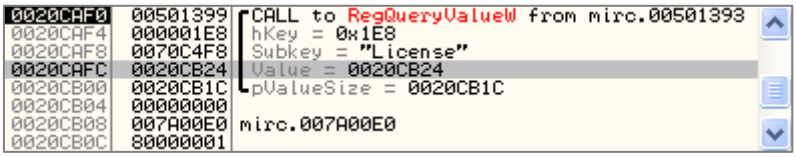
Vamos a Help -> Register y nos registramos otra vez con el Name/Serial que tenemos, hecho esto reiniciamos Olly, en el Entry Point activamos los Patches y además colocamos BP's en todos ellos, damos RUN y aparece el diálogo de activación sin parar en los BP's.

Si miramos el registro ahora veremos NO que están todas las claves:

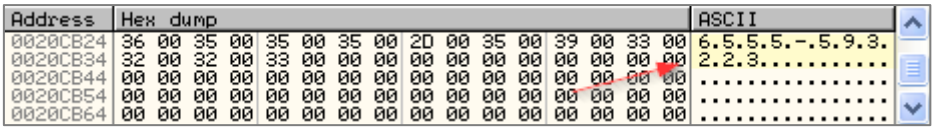


Faltan validated2, validated3, validated4 y validated5.

Está claro que de alguna manera solicita confirmación de registro y que de alguna manera debe mirar si estamos registrados, como los datos los tiene en el registro colocaremos 2 BP's, uno en la función RegQueryValueA y otro en RegQueryValueW (estamos en el entry point con los patches activados), <F9>..



Para en RegQueryValueW para leer el contenido de la clave License, que dejará en en 0x20CB24:



Allí está en el búfer, traceamos con <F8> desde aquí y retornamos a esta rutina:

```
00502B50  >  68 F40F7000   push    00700FF4
00502B55  .  6A 11         push    0x11
00502B57  .  6A 00         push    0x0
00502B59  .  68 F8C47000   push    0070C4F8                    UNICODE "License"
00502B5E  .  56            push    esi
00502B5F  .  E8 4CE6FFFF   call    005011B0
00502B64  .  8BE8          mov     ebp, eax
00502B66  .  6A 00         push    0x0
00502B68  .  8BFD          mov     edi, ebp
00502B6A  .  83CF 10       or      edi, 0x10
00502B6D  .  57            push    edi
00502B6E  .  8D4424 2C     lea     eax, dword ptr [esp+0x2C]
00502B72  .  50            push    eax
00502B73  .  68 F8C47000   push    0070C4F8                    UNICODE "License"
00502B78  .  56            push    esi
00502B79  .  E8 32E6FFFF   call    005011B0
00502B7E  .  83C4 28       add     esp, 0x28
00502B81  .  85C0          test    eax, eax
00502B83  .v 74 36         je      short 00502BBB
00502B85  .  6A 00         push    0x0
00502B87  .  57            push    edi
00502B88  .  8D8C24 E8200  lea     ecx, dword ptr [esp+0x20E8]
00502B8F  .  51            push    ecx
00502B90  .  68 08C57000   push    0070C508                    UNICODE "UserName"
00502B95  .  56            push    esi
00502B96  .  E8 15E6FFFF   call    005011B0
00502B9B  .  83C4 14       add     esp, 0x14
00502B9E  .  85C0          test    eax, eax
00502BA0  .v 74 19         je      short 00502BBB
00502BA2  .  8D5424 10     lea     edx, dword ptr [esp+0x10]
00502BA6  .  52            push    edx
00502BA7  .  8D8424 E4200  lea     eax, dword ptr [esp+0x20E4]
00502BAE  .  50            push    eax
00502BAF  .  E8 5CDFFFFF   call    00500B10
00502BB4  .  83C4 08       add     esp, 0x8
00502BB7  .  85C0          test    eax, eax
00502BB9  .v 75 64         jnz     short 00502C1F              // ** si no salta borra keys
00502BBB  >  6A 10         push    0x10
00502BBD  .  68 F8C47000   push    0070C4F8                    UNICODE "License"
00502BC2  .  56            push    esi
00502BC3  .  E8 B8D9FFFF   call    00500580
```

Como he puesto en el comentario de la imagen, si no salta en el JNZ de 0x502BB9 borrará las claves del registro, pero este salto no es problema como se puede comprobar:

```
00502BA7  .  8D8424 E4200  lea     eax, dword ptr [esp+0x20E4]
00502BAE  .  50            push    eax
00502BAF  .  E8 5CDFFFFF   call    <VALIDA_SERIAL>
00502BB4  .  83C4 08       add     esp, 0x8
00502BB7  .  85C0          test    eax, eax
00502BB9  .v 75 64         jnz     short 00502C1F              // ** si no salta borra keys
00502BBB  >  6A 10         push    0x10
00502BBD  .  68 F8C47000   push    0070C4F8                    UNICODE "License"
00502BC2  .  56            push    esi
00502BC3  .  E8 B8D9FFFF   call    00500580
00502BC8  .  6A 10         push    0x10
00502BCA  .  68 08C57000   push    0070C508                    UNICODE "UserName"
00502BCF  .  56            push    esi
00502BD0  .  E8 ABD9FFFF   call    00500580
00502BD5  .  6A 50         push    0x50
00502BD7  .  68 E4C47000   push    0070C4E4                    UNICODE "Validated"
00502BDC  .  56            push    esi
```

El salto se produce, el call anterior se encarga de validar el serial que es bueno por eso salta, debe de haber otra comprobación, seguiremos traceando desde aquí, llegamos a un bucle donde hay dos call:

```
00502CAB  .  56            push    esi
00502CAC  .  E8 FFE4FFFF   call    005011B0                    // ** Lee Validated
00502CB1  .  83C4 14       add     esp, 0x14
00502CB4  .  85C0          test    eax, eax
00502CB6  .v 74 42         je      short 00502CFA
00502CB8  .  6A 03         push    0x3
00502CBA  .  8D8C24 E4200  lea     ecx, dword ptr [esp+0x20E4]
00502CC1  .  51            push    ecx
00502CC2  .  8D5424 18     lea     edx, dword ptr [esp+0x18]
00502CC6  .  52            push    edx
00502CC7  .  E8 F4E2FFFF   call    00500FC0
00502CCC  .  83C4 0C       add     esp, 0xC
00502CCF  .  85C0          test    eax, eax
00502CD1  .v 74 27         je      short 00502CFA
00502CD3  .  53            push    ebx
```

El primero de ellos obtuvo el contenido de la clave Validated y la dejó en 0x20CB24 (call de 0x502CAC), el segundo de éstos en el que nos encontramos ha recibido 3 parámetros: el valor 0x3, un puntero al serial y un puntero a la string "2731c26fbc02CAA7A1cE34A33bB1be830B2694e5eda6a986121212redirect=7878" que es el contenido de la clave Validated.

Dentro del segundo call (en 0x502CC7) hace muchísimas llamadas tipo kernel32.GetDiskFreeSpaceW, RPCRT4.UuidCreateSequential, entre otras, después llega a este subcall:

```
005010D8  .  51              push   ecx
005010D9  .  E8 F2EAFFFF     call   004FFBD0
005010DE  .  83C4 24         add    esp, 0x24
005010E1  .  85C0            test   eax, eax
005010E3  .v 75 6F           jnz    short 00501154                    // ** BUENO
005010E5  >  F6C3 02         test   bl, 0x2
005010E8  .v 0F84 8300000    je     00501171
005010EE  .  6A 02           push   0x2
005010F0  .  8D5424 10       lea    edx, dword ptr [esp+0x10]
005010F4  .  68 D0200000     push   0x20D0
```

Marcamos el JNZ como bueno porque de tomarlo retornamos del call con EAX==0x1 (al contrario de lo que hace ahora que es EAX==0x0), llegado un punto salta a esta función:

```
004FFC67  .  53              push   ebx
004FFC68  .  8D4424 28       lea    eax, dword ptr [esp+0x28]
004FFC6C  .  50              push   eax
004FFC6D  .  895C24 2C       mov    dword ptr [esp+0x2C], ebx
004FFC71  .  FF15 34F06F0    call   dword ptr [<&ADVAPI32.CryptAc  ADVAPI32.CryptAcquireContextW
004FFC77  .  85C0            test   eax, eax
004FFC79  .v 0F84 AA01000    je     004FFE29
004FFC7F  .  68 FF070000     push   0x7FF
004FFC84  .  8D8C24 3D080    lea    ecx, dword ptr [esp+0x83D]
004FFC8B  .  53              push   ebx
```

Me hace recordar a L0ngin0s que no hará mucho tuvo un encontronazo con ella (un saludo amigo :D), si miramos los registros encontramos unas strings super largas

```
EAX 00209A04
ECX D09F0C00
EDX 016EA131 ASCII "21212redirect=7878"
EBX 00000000
ESP 002099D8
EBP 016E8010 ASCII
    "MIGdMA0GCSqGSIb3DQEBAQUAA4GLADCBhwKBgQCyUNXfZ1IGbfPuay+7dDCF+uBUKygsMGZigpmiHWUz3Pfnav
    2ST0wBaxNxAeWuTSW1sIhmFAYA/bpjlGyrgohogtoUYbbJ56enam2Wplmr79O1f27wPc29mVAg932zA8lz2WR8D
    qnzYsuWZ383mCVQ65oVU9QjXdc2GSlZqQNUpwIBEQ=="
ESI 0020AA5C UNICODE "2731c26fbc02CAA7A1cE34A33bB1be830B2694e5eda6a986"
EDI 016EA130 ASCII "121212redirect=7878"
```

Sigue con llamadas a CryptStringToBinaryA, CryptDecodeObjectEx, CryptImportPublic, CryptCreateHash, CryptHashData, CryptStringToBinaryA (por segunda vez y falla):

```
004FFD9A  .  6A 01           push   0x1
004FFD9C  .  52              push   edx
004FFD9D  .  57              push   edi
004FFD9E  .  C74424 2C 00    mov    dword ptr [esp+0x2C], 0x800
004FFDA6  .  FFD6            call   esi
004FFDA8  .  85C0            test   eax, eax
004FFDAA  .v 74 50           je     short 004FFDFC
004FFDAC  .  8B7424 10       mov    esi, dword ptr [esp+0x10]
004FFDB0  .  8D46 FF         lea    eax, dword ptr [esi-0x1]
004FFDB3  .  33C9            xor    ecx, ecx
004FFDB5  .  3BC3            cmp    eax, ebx
004FFDB7  .v 7E 1F           jle    short 004FFDD8
004FFDB9  .  8DA424 00000    lea    esp, dword ptr [esp]
004FFDC0  >  8A5C04 38       mov    bl, byte ptr [esp+eax+0x38]
004FFDC4  .  8A540C 38       mov    dl, byte ptr [esp+ecx+0x38]
004FFDC8  .  885C0C 38       mov    byte ptr [esp+ecx+0x38], bl
004FFDCC  .  885404 38       mov    byte ptr [esp+eax+0x38], dl
004FFDD0  .  41              inc    ecx
004FFDD1  .  48              dec    eax
004FFDD2  .  3BC8            cmp    ecx, eax
004FFDD4  .^ 7C EA           jl     short 004FFDC0
004FFDD6  .  33DB            xor    ebx, ebx
004FFDD8  >  8B4424 1C       mov    eax, dword ptr [esp+0x1C]
004FFDDC  .  8B5424 14       mov    edx, dword ptr [esp+0x14]
004FFDE0  .  53              push   ebx
004FFDE1  .  53              push   ebx
004FFDE2  .  50              push   eax
004FFDE3  .  56              push   esi
004FFDE4  .  8D4C24 48       lea    ecx, dword ptr [esp+0x48]
004FFDE8  .  51              push   ecx
004FFDE9  .  52              push   edx
004FFDEA  .  FF15 08F06F0    call   dword ptr [<&ADVAPI32.CryptVe  ADVAPI32.CryptVerifySignatureW
004FFDF0  .  85C0            test   eax, eax
004FFDF2  .v 74 08           je     short 004FFDFC
004FFDF4  .  C74424 24 01    mov    dword ptr [esp+0x24], 0x1      // ** exito
004FFDFC  >  8B4424 14       mov    eax, dword ptr [esp+0x14]
004FFE00  .  50              push   eax
```

Ese JE hace que saltemos por encima del 'mov dword ptr [esp+0x24], 0x1' y nos saque de la función pasando por CryptDestroyHash, CryptDestroyKey y CryptReleaseContext.

El valor de retorno se lo lleva EAX justo aquí:

```
004FFE12  > 8B5424 20      mov     edx, dword ptr [esp+0x20]
004FFE16  . 52             push    edx                                hMemory
004FFE17  . FF15 04F26F0   call    dword ptr [<&KERNEL32.LocalFr      LocalFree
004FFE1D  > 8B4424 18      mov     eax, dword ptr [esp+0x18]
004FFE21  . 53             push    ebx
004FFE22  . 50             push    eax
004FFE23  . FF15 28F06F0   call    dword ptr [<&ADVAPI32.CryptRe      ADVAPI32.CryptReleaseContext
004FFE29  > 8B4424 24      mov     eax, dword ptr [esp+0x24]          // ** ret_value
004FFE2D  . 5D             pop     ebp
004FFE2E  . 5F             pop     edi
004FFE2F  . 5E             pop     esi
004FFE30  . 5B             pop     ebx
004FFE31  . 8B8C24 28100   mov     ecx, dword ptr [esp+0x1028]
004FFE38  . 33CC           xor     ecx, esp
004FFE3A  . E8 F5E80B00    call    005BE734
004FFE3F  . 81C4 2C10000   add     esp, 0x102C
004FFE45  . C3             ret
004FFE46  > 8B8C24 34100   mov     ecx, dword ptr [esp+0x1034]
Stack ss:[00209A10]=00000000
eax=00000001
```

Si fuera igual a 0x1 la cosa iría bien, pues al retornar lo comprueba:

```
005010D8  . 51             push    ecx
005010D9  . E8 F2EAFFFF    call    004FFBD0
005010DE  . 83C4 24        add     esp, 0x24
005010E1  . 85C0           test    eax, eax
005010E3  .v 75 6F         jnz     short 00501154                     // ** BUENO
005010E5  > F6C3 02        test    bl, 0x2
005010E8  .v 0F84 8300000  je      00501171
005010EE  . 6A 02          push    0x2
005010F0  . 8D5424 10      lea     edx, dword ptr [esp+0x10]
```

Después lo vuelve a intentar:

```
00501147  . 51             push    ecx
00501148  . E8 83EAFFFF    call    004FFBD0
0050114D  . 83C4 24        add     esp, 0x24
00501150  . 85C0           test    eax, eax
00501152  .v 74 1D         je      short 00501171                     // ** MALO
00501154  > 5B             pop     ebx
00501155  . 5F             pop     edi
00501156  . B8 01000000    mov     eax, 0x1                           // ** flag de exito
0050115B  . 5E             pop     esi
0050115C  . 8B8C24 70620   mov     ecx, dword ptr [esp+0x6270]
00501163  . 33CC           xor     ecx, esp
00501165  . E8 CAD50B00    call    005BE734
0050116A  . 81C4 7462000   add     esp, 0x6274
00501170  . C3             ret
00501171  > 8B8C24 7C620   mov     ecx, dword ptr [esp+0x627C]
00501178  . 5B             pop     ebx
00501179  . 5F             pop     edi
0050117A  . 5E             pop     esi
0050117B  . 33CC           xor     ecx, esp
0050117D  . 33C0           xor     eax, eax                           // ** flag de error
0050117F  . E8 B0D50B00    call    005BE734
00501184  . 81C4 7462000   add     esp, 0x6274
0050118A  . C3             ret
```

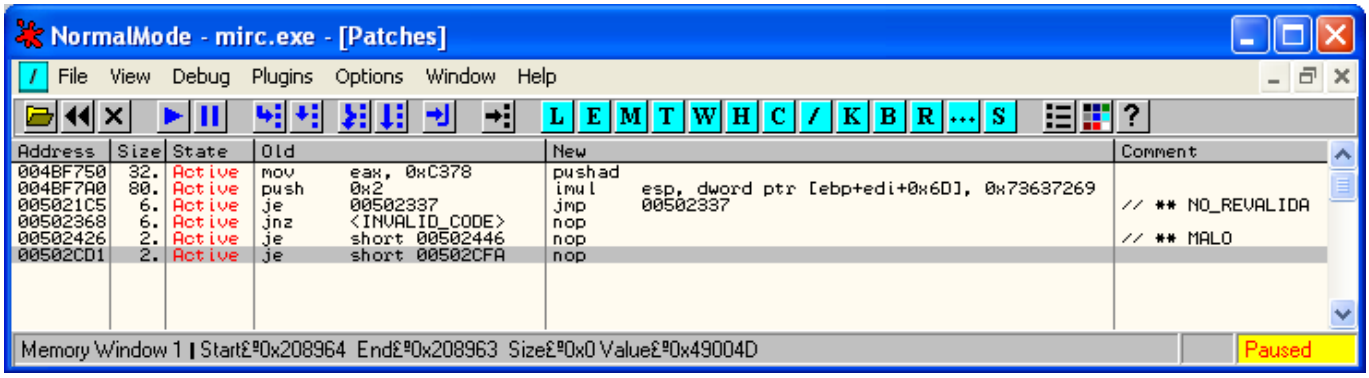
Obtiene el mismo resultado y sale del call con flag de error (EAX==0x0) que se valida al salir de éste:

```
00502CC6  . 52             push    edx
00502CC7  . E8 F4E2FFFF    call    00500FC0                           // ** Crypt..
00502CCC  . 83C4 0C        add     esp, 0xC
00502CCF  . 85C0           test    eax, eax
00502CD1  .v 74 27         je      short 00502CFA
00502CD3  . 53             push    ebx
00502CD4  . 68 C4000000    push    0xC4
00502CD9  . 53             push    ebx
00502CDA  . 68 BE000000    push    0xBE
00502CDF  . 891D 44B3820   mov     dword ptr [0x82B344], ebx
00502CE5  . 891D 8CCD810   mov     dword ptr [0x81CD8C], ebx
00502CEB  . E8 70980100    call    0051C560
00502CF0  . 83C4 08        add     esp, 0x8
00502CF3  . 50             push    eax                                hMenu
00502CF4  . FF15 E0F66F0   call    dword ptr [<&USER32.DeleteMe       DeleteMenu
00502CFA  > 47             inc     edi
00502CFB  . 83FF 05        cmp     edi, 0x5
00502CFE  .^ 0F8E 5CFFFFF  jle     00502C60
00502D04  . E8 37ECFFFF    call    00501940
00502D09  > 8B8C24 80620   mov     ecx, dword ptr [esp+0x6280]        Default case of switch 00502BE5
00502D10  . 5F             pop     edi
```

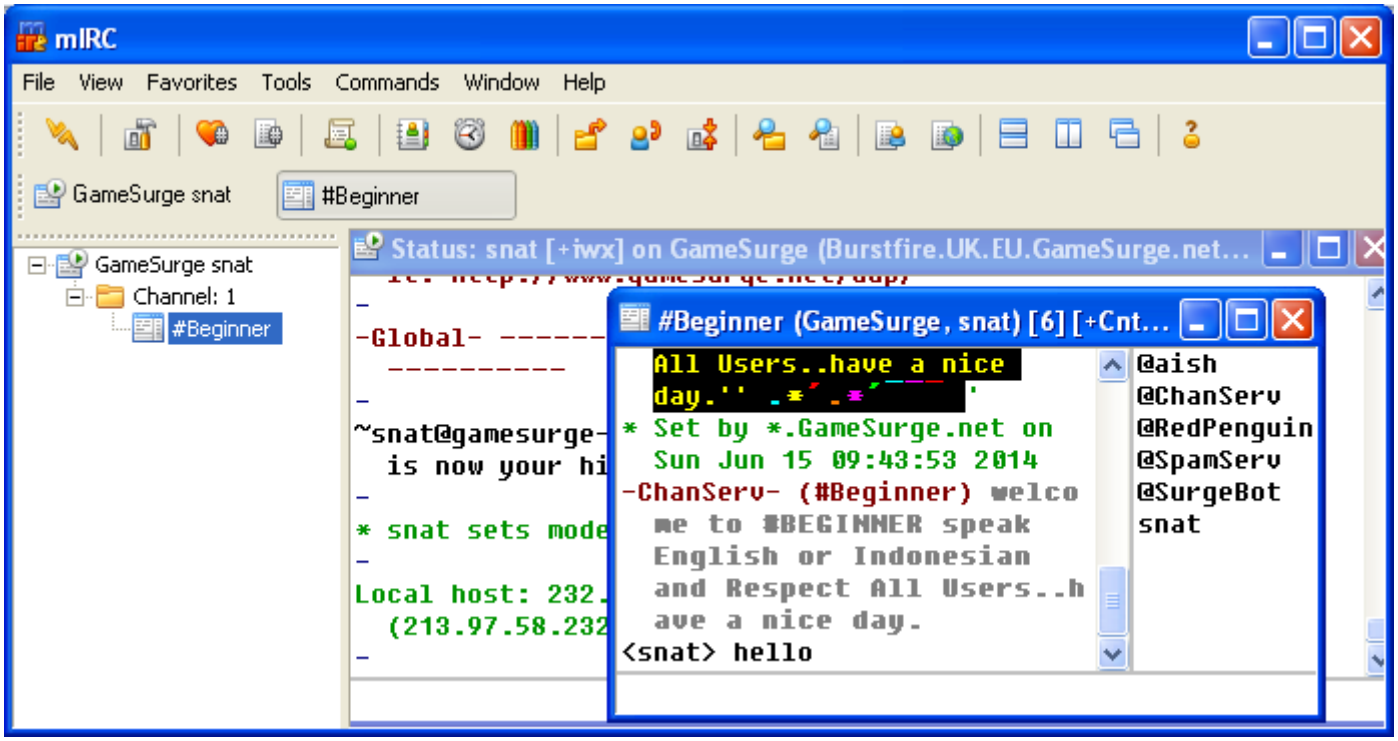
Tenemos varias opciones, pelearnos con la Crypt o modificar el salto. La mejor opción, sin duda, es la primera, pero en este caso igual nos va a dar porque ya hemos modificado el binario y lo más rápido es modificarlo un poco más, podemos modificar la rutina internamente para que retorne EAX==0x1 que es bastante fiable o NOPear el JE, esta última es menos intrusiva y probaremos a ver si nos llega:

```
00502CC6  .  52                push    edx
00502CC7  .  E8 F4E2FFFF       call    00500FC0                 // ** Crypt..
00502CCC  .  83C4 0C           add     esp, 0xC
00502CCF  .  85C0              test    eax, eax
00502CD1     90                nop
00502CD2     90                nop
00502CD3  .  53                push    ebx
00502CD4  .  68 C4000000       push    0xC4
00502CD9  .  53                push    ebx
00502CDA  .  68 BE000000       push    0xBE
00502CDF  .  891D 44B3820(     mov     dword ptr [0x82B344], ebx
00502CE5  .  891D 8CCD810(     mov     dword ptr [0x81CD8C], ebx
00502CEB  .  E8 70980100       call    0051C560
00502CF0  .  83C4 08           add     esp, 0x8
00502CF3  .  50                push    eax                      hMenu
00502CF4  .  FF15 E0F66F0(     call    dword ptr [<&USER32.DeleteMe  DeleteMenu
00502CFA  >  47                inc     edi
00502CFB  .  83FF 05           cmp     edi, 0x5
00502CFE  .^ 0F8E 5CFFFFF(     jle     00502C60
00502D04  .  E8 37ECFFFF       call    00501940
00502D09  >  8B8C24 80620(     mov     ecx, dword ptr [esp+0x6280]   Default case of switch 00502BE5
```

Salvamos este cambio, junto con todos los demás al ejecutable.



Esos son todas las modificaciones. Probamos el programa al vuelo con <F9> a ver si corre:



Funciona bien y podemos hablar en un canal al que nos acabamos de unir, estarán alucinando porque no nos conocen xDD

Cerramos Olly y cargamos el exe modificado, demos RUN y…

```
Process terminated, exit code 0                                    Terminated
```

No funciona.

Lo más probable es que haya un control de integridad en el ejecutable, para saberlo habrá que depurar un poco más y mirar de parar en las APIs más comunes, como estas:

```
0046CD0B  .   6A 01          push    0x1                          ShareMode = FILE_SHARE_READ
0046CD0D  .   68 00000080    push    0x80000000                   Access = GENERIC_READ
0046CD12  .   C701 0000000   mov     dword ptr [ecx], 0x0
0046CD18  .   50             push    eax                          FileName
0046CD19  .   C745 00 0000   mov     dword ptr [ebp], 0x0
0046CD20  .   C74424 30 00   mov     dword ptr [esp+0x30],
0046CD28  .   FF15 18F36F0   call    dword ptr [<&KERNEL32.       CreateFileW
0046CD2E  .   8BF0           mov     esi, eax
0046CD30  .   85F6           test    esi, esi
0046CD32  .v  0F84 9C00000   je      0046CDD4
0046CD38  .   57             push    edi
0046CD39  .   6A 00          push    0x0                          MapName = NULL
0046CD3B  .   6A 00          push    0x0                          MaximumSizeLow = 0x0
0046CD3D  .   6A 00          push    0x0                          MaximumSizeHigh = 0x0
0046CD3F  .   6A 02          push    0x2                          Protection = PAGE_READONLY
0046CD41  .   6A 00          push    0x0                          pSecurity = NULL
0046CD43  .   56             push    esi                          hFile
0046CD44  .   FF15 1CF36F0   call    dword ptr [<&KERNEL32.       CreateFileMappingW
0046CD4A  .   8BF8           mov     edi, eax
0046CD4C  .   897C24 10      mov     dword ptr [esp+0x10],
0046CD50  .   85FF           test    edi, edi
0046CD52  .v  74 78          je      short 0046CDCC
0046CD54  .   53             push    ebx
0046CD55  .   6A 00          push    0x0                          MapSize = 0x0
0046CD57  .   6A 00          push    0x0                          OffsetLow = 0x0
0046CD59  .   6A 00          push    0x0                          OffsetHigh = 0x0
0046CD5B  .   6A 04          push    0x4                          AccessMode = FILE_MAP_READ
0046CD5D  .   57             push    edi                          hMapObject
0046CD5E  .   FF15 20F36F0   call    dword ptr [<&KERNEL32.       MapViewOfFile
0046CD64  .   8BD8           mov     ebx, eax
```

Si miramos la pila vemos los parámetros de CreateFileW:

```
0021BCF8  0021BD4C  FileName = "C:\Archivos de programa\mIRC 7.43\mirc_final.exe"
0021BCFC  80000000  Access = GENERIC_READ
0021BD00  00000001  ShareMode = FILE_SHARE_READ
0021BD04  00000000  pSecurity = NULL
0021BD08  00000003  Mode = OPEN_EXISTING
0021BD0C  00000000  Attributes = 0
0021BD10  00000000  hTemplateFile = NULL
0021BD14  00000021
```

Esta función se utiliza para crear un archivo o bien abrirlo en caso de existir, esto dependerá de los parámetros que se le pasen, en este caso OPEN_EXISTING indica que lo abrirá en caso de existir y GENERIC_READ que lo hará para lectura.

El objetivo de las dos siguientes llamadas CreateFileMappingW retornará el manejador de una sección creada para el archivo solicitado (especificado por el handle de la llamada), es decir, CreateFile retorna un handle de archivo (en caso de éxito), y éste handle se le pasa a CreateFileMapping (entre otros parámetros), si la función tiene éxito retorna el manejador a una nueva sección, en este caso retorna EAX==0x128, veamos la sección en la ventana de Handles de Olly:

Handle	Type	Refs	Access	T	Info	Name
00000104	IoCompletion	2.	001F0003			
00000108	IoCompletion	5.	001F0003			
0000010C	IoCompletion	5.	001F0003			
00000110	IoCompletion	5.	001F0003			
00000114	Event	2.	001F0003			
00000118	Port	2.	001F0001			
0000011C	File	2.	00120089		Size 462464	c:\Archivos de programa\mIRC 7.43\mirc_final.exe
00000120	Key	2.	000F003F			HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\E
00000124	Semaphore	17.	001F0003		Count 598.	\BaseNamedObjects\shell.{7CB834F0-527B-11D2-9D1F-0000F805CA57
00000128	Section	2.	000F0005			

Y este manejador (retornado por CreateFileMapping) es el que se le pasa a MapViewOfFile:

```
0046CD54  .  53            push    ebx
0046CD55  .  6A 00         push    0x0                 MapSize = 0x0
0046CD57  .  6A 00         push    0x0                 OffsetLow = 0x0
0046CD59  .  6A 00         push    0x0                 OffsetHigh = 0x0
0046CD5B  .  6A 04         push    0x4                 AccessMode = FILE_MAP_READ
0046CD5D  .  57            push    edi                 hMapObject = 00000128 (window)
0046CD5E  .  FF15 20F36F0  call    dword ptr [<&KERNEL32.  MapViewOfFile
0046CD64  .  8BD8          mov     ebx, eax
0046CD66  .  85DB          test    ebx, ebx
```

Al retornar de la función en EAX estará la dirección base al archivo mapeado en memoria, en este caso deja EAX==0x01260000, si miramos allí vemos, efectivamente, el archivo en memoria:

```
Address   Hex dump                                          ASCII
01260000  4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00  MZÉ.♥...♦...  ..
01260010  B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  ¸.......@.......
01260020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
01260030  00 00 00 00 00 00 00 00 00 00 00 00 E0 00 00 00  ............à...
01260040  0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68  ♫▼º♫.┤.=!¸☺L=!Th
01260050  69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F  is program canno
01260060  74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20  t be run in DOS
01260070  6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00  mode....$.......
01260080  9A 38 3E CD DE 59 50 9E DE 59 50 9E DE 59 50 9E  Ü8>=ìYPxìYPxìYPx
01260090  DE 59 50 9E CA 59 50 9E D7 21 C5 9E E1 59 50 9E  ìYPx╩YPx╫!+xßYPx
012600A0  D7 21 D3 9E 0B 5B 50 9E D7 21 C3 9E C1 59 50 9E  ╫!╙x♂[Px╫!├x┴YPx
012600B0  DE 59 51 9E 8D 5B 50 9E D7 21 D4 9E A7 58 50 9E  ìYQxì[Px╫!╘x§XPx
012600C0  C0 0B C4 9E DF 59 50 9E D7 21 C1 9E DF 59 50 9E  └♂─x▀YPx╫!┴x▀YPx
012600D0  52 69 63 68 DE 59 50 9E 00 00 00 00 00 00 00 00  RichìYPx........
012600E0  50 45 00 00 4C 01 05 00 B5 87 BF 55 00 00 00 00  PE..L♪♣.µçïU....
012600F0  00 00 00 00 E0 00 02 01 0B 01 09 00 00 D4 2F 00  ....à.☻♪○♂○...Ô/.
01260100  00 A2 16 00 00 00 00 00 90 89 1C 00 00 10 00 00  .¢▬.....Éë∟..►..
```

Se ve claramente que es el contenido del exe, a partir de aquí llamará a la función CheckSumMappedFile de la librería imagehlp.dll, ésta retornará un puntero al IMAGE_NT_HEADERS (donde está la cadena 'PE'), pero más importante son los parámetros que se le pasan:

HeaderSum [out]

A pointer to a variable that receives the original checksum from the image file, or zero if there is an error.

CheckSum [out]

A pointer to the variable that receives the computed checksum.

En los que retornará dos punteros, uno al checksum del archivo (ubicado en el header del mismo) y otro al calculado, si miremos qué retornó:

```
Address   Hex dump                                          ASCII
0021BD44  50 B0 46 00 92 D9 46 00 43 00 3A 00 5C 00 41 00  P▒F.╝┘F.C.:.\.A.
0021BD54  72 00 63 00 68 00 69 00 76 00 6F 00 73 00 20 00  r.c.h.i.v.o.s. .
0021BD64  64 00 65 00 20 00 70 00 72 00 6F 00 67 00 72 00  d.e. .p.r.o.g.r.
0021BD74  61 00 6D 00 calculado 00 valido 3 52 00 43 00  a.m.a.\.m.I.R.C.
0021BD84  20 00 37 00 2E 00 34 00 33 00 5C 00 6D 00 69 00   .7...4.3.\.m.i.
0021BD94  72 00 63 00 5F 00 66 00 69 00 6E 00 61 00 6C 00  r.c._.f.i.n.a.l.
0021BDA4  2E 00 65 00 78 00 65 00 00 00 00 00 00 00 00 00  ..e.x.e.........
0021BDB4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0021BDC4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0021BDD4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

El valor válido lo encontramos en la cabecera del archivo:

```
Address   Hex dump                                          ASCII
00400120  05 00 00 00 00 00 00 00 05 00 00 00 00 00 00 00  ♣.......♣.......
00400130  00 30 64 00 00 04 00 00 92 D9 46 00 02 00 40 81  .0d..♦..╝┘F.☻.@ü
00400140  80 84 1E 00 00 10 00 00 00 00 10 00 00 10 00 00  Çä▲..►....►..►..
00400150  00 00 00 00 10 00 00 00 00 00 00 00 00 00 00 00  ....►...........
00400160  CC F9 37 00 40 01 00 00 00 E0 56 00 [ valido 3  ╠╙7.@☺...àV.ìk..
00400170  00 00 00 00 00 00 00 00 00 7A 46 00 00 17 00 00  .........zF..‡..
00400180  00 50 61 00 E4 65 02 00 00 00 00 00 00 00 00 00  .Pa.äe☻.........
00400190  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004001A0  00 00 00 00 00 00 00 00 18 22 37 00 40 00 00 00  ........↑"7.@...
004001B0  00 00 00 00 00 00 00 00 00 F0 2F 00 40 09 00 00  .........≡/.@...
```

Que no coincide con el calculado 0x46B050.

Esta comprobación se realiza al salir del call:

```
0046CE3C  .  83C4 1C        add     esp, 0x1C
0046CE3F  .  85C0           test    eax, eax
0046CE41  .v 74 25          je      short 0046CE68
0046CE43  .  8B4424 04      mov     eax, dword ptr [esp+0x4]
0046CE47  .  3B0424         cmp     eax, dword ptr [esp]          mirc_fin.0046B050
0046CE4A  .v 74 1C          je      short 0046CE68
0046CE4C  .  F7D8           neg     eax
0046CE4E  .  1BC0           sbb     eax, eax
0046CE50  .  83C0 03        add     eax, 0x3
0046CE53  .  8B8C24 D8200   mov     ecx, dword ptr [esp+0x20D8]
0046CE5A  .  33CC           xor     ecx, esp
0046CE5C  .  E8 D3181500    call    005BE734
0046CE61  .  81C4 DC20000   add     esp, 0x20DC
0046CE67  .  C3             ret
Stack ss:[0021BD44]=0046B050 (mirc_fin.0046B050)
eax=0046D992 (mirc_fin.0046D992)
```

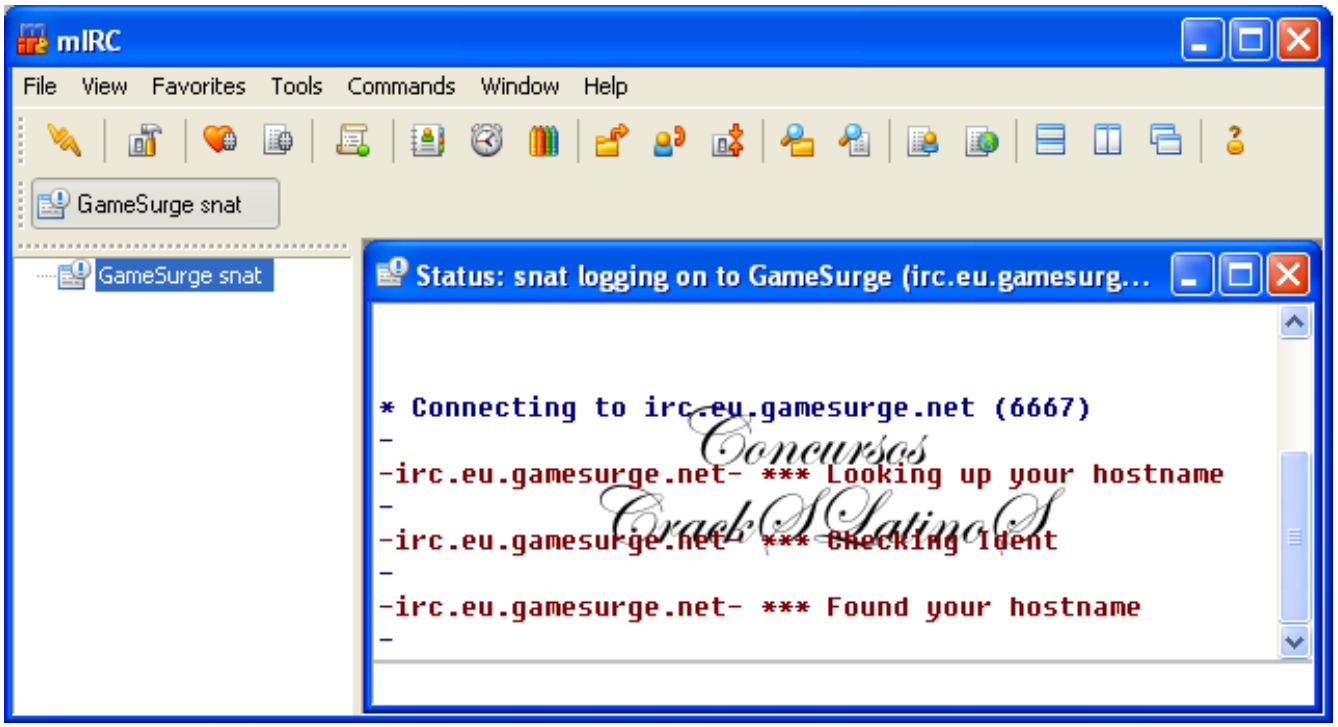
Y nos saca fuera con EAX==0x2 (flag de error).

Hay varias formas de derrotar esto, una que va muy bien es cambiar el path en la llamada a CreateFile y que apunte al archivo original sin modificar.

En este caso, y ya que hemos parcheado el binario JMPeamos el salto:

```
0046CE41  .v 74 25          je      short 0046CE68
0046CE43  .  8B4424 04      mov     eax, dword ptr [esp+0x4]
0046CE47  .  3B0424         cmp     eax, dword ptr [esp]          mirc_fin.0046B050
0046CE4A  .v EB 1C          jmp     short 0046CE68
0046CE4C  .  F7D8           neg     eax
0046CE4E  .  1BC0           sbb     eax, eax
0046CE50  .  83C0 03        add     eax, 0x3
0046CE53  .  8B8C24 D8200   mov     ecx, dword ptr [esp+0x20D8]
0046CE5A  .  33CC           xor     ecx, esp
0046CE5C  .  E8 D3181500    call    005BE734
0046CE61  .  81C4 DC20000   add     esp, 0x20DC
0046CE67  .  C3             ret
0046CE68  >  8B8C24 D8200   mov     ecx, dword ptr [esp+0x20D8]
0046CE6F  .  33CC           xor     ecx, esp
0046CE71  .  B8 01000000    mov     eax, 0x1
0046CE76  .  E8 B9181500    call    005BE734
0046CE7B  .  81C4 DC20000   add     esp, 0x20DC
0046CE81  .  C3             ret
0046CE82     CC             int3
```

Salvamos los cambios y lo probamos..



Funciona perfecto !

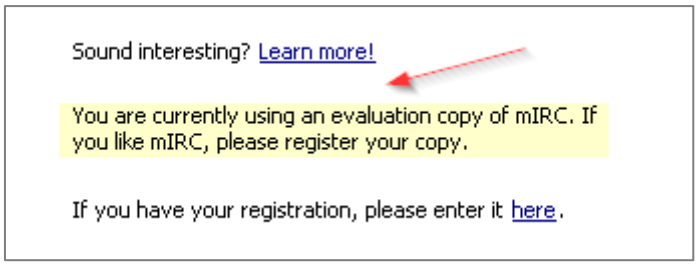
7. Tirando de Recursos

Hasta ahora hemos visto cómo solucionar el esquema de protección Trial de 30 días desbloqueable por serial y con validación on-line. Con todos los algoritmos de cifrado y/o encriptación que implementa no ha podido evitar que nos registremos localmente (sin validar online).

Realmente no sé si con esto es suficiente y habría que ir más allá (forzando esta validación), lo que sí sé es que vamos a ver una manera alternativa de encontrar el punto caliente del programa sin pasar por la ventana de registro, hay esquemas de protección con keyfile que no disponen de esta opción por ejemplo.

Borramos el archivo mirc_final.exe anterior y ejecutamos mirc.exe (original), se quejará de la licencia falsa del registro mostrando la ventana anterior de validación, pulsamos <Cancel> y ya estamos listos.

Vamos a la ventanita About y nos fijamos las strings que hay allí:



Abrimos Resource Hacker Plus y cargamos el ejecutable (mirc.exe), hacemos una búsqueda de la cadena:

```
STRINGTABLE
LANGUAGE LANG_ENGLISH, SUBLANG_ENGLISH_UK
{
101,   "%d day remaining."
102,   "%d days remaining."
103,   "You are using a licensed copy of mIRC that is registered to the following indi
105,   "Your evaluation copy of mIRC has expired. To continue using mIRC, please regis
106,   "You are currently using an evaluation copy of mIRC. If you like mIRC, please r
107,   "Exit"
108,   "Continue"
109,   "To register mIRC, click on the register button below."
110,   "If you like mIRC, and use it often, please register your copy."
111,   "To register mIRC, click on the register button below."
}
```

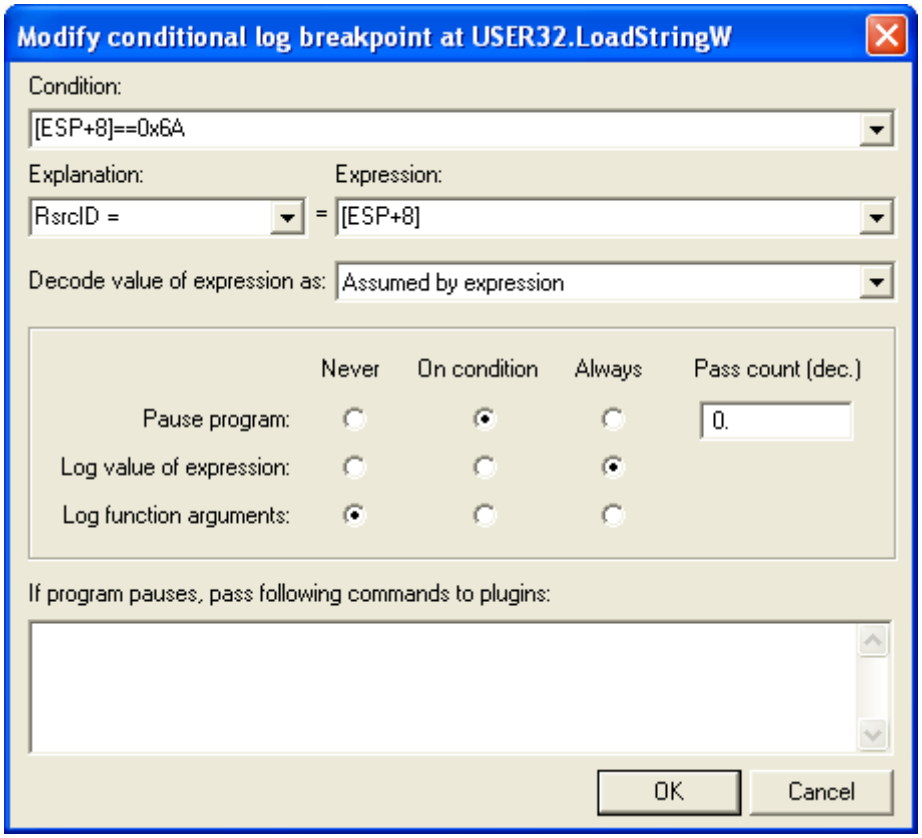
Nos interesa el ID que es 106 decimal.

Lo pasamos a hexadecimal:

- ✓ `106d == 0x6A`

Este valor 0x6A es el que será usado por parte del programa a la hora de cargar la cadena en el cuadro de diálogo, sabemos que esta string está en la STRING_TABLE y que en algún momento, durante la ejecución del programa en modo Trial deberá acceder a ella.

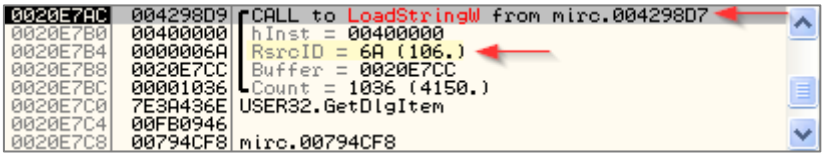
Habitualmente se utiliza la función LoadStringA/W, como estamos viendo que trabaja con strings UCNICODE nos centramos en LoadStringW, en el entry point vamos a esta api <Ctrl+G> + su nombre y colocamos un BP Conditional Log:



Lo explicamos:

- ✓ **Condition: [ESP+8]==0x6A** : Para que evalúe esta condición, en [ESP+8] está el parámetro de identificación de la cadena cargada (nuestro ID vigilado 0x6A)
- ✓ **Explanation: RsrcID=[ESP+8]** : Para que loguee el valor del ID.
- ✓ **Pause Program: On Condition** : El programa se pausará cuando [ESP+8]==0x6A
- ✓ **Log value of expression: Always** : Se escribirán en el log el valor de [ESP+8] en cada parada.

Dejamos correr el programa y Olly para en el BP, miremos la pila:



Dos detalles interesantes:

1. el ID es el 0x6A (Evaluation..)
2. el origen de la llamada 0x4298D7

Traceamos con <F8> hasta salir del call.

Un par de RETs después caemos aquí:

```
00401B79  .  8908          mov     dword ptr [eax], ecx
00401B7B  >  8B3D 34F86F0  mov     edi, dword ptr [<&USER32.GetDlg  USER32.GetDlgItem
00401B81  .  68 C8000000   push    0xC8                             ControlID = C8 (200.)
00401B86  .  56            push    esi                              hWnd
00401B87  .  FFD7          call    edi                              GetDlgItem
00401B89  .  85C0          test    eax, eax
00401B8B  .v 0F85 F401000  jnz     00401D85
00401B91  .  6A 02         push    0x2
00401B93  .  8D5424 34     lea     edx, dword ptr [esp+0x34]
00401B97  .  68 D0200000   push    0x20D0
00401B9C  .  52            push    edx
00401B9D  .  E8 0EDD1900   call    0059F8B0
00401BA2  .  8D4424 3C     lea     eax, dword ptr [esp+0x3C]
00401BA6  .  50            push    eax
00401BA7  .  6A 6A         push    0x6A                             // ** Unregistered evaluation
00401BA9  .  E8 627C0200   call    00429810
00401BAE  .  83C4 14       add     esp, 0x14
00401BB1  .  8D4C24 30     lea     ecx, dword ptr [esp+0x30]
00401BB5  .  51            push    ecx                              lParam
00401BB6  .  6A 00         push    0x0                              wParam = 0x0
00401BB8  .  6A 0C         push    0xC                              Message = WM_SETTEXT
00401BBA  .  68 A6000000   push    0xA6                             ControlID = A6 (166.)
00401BBF  .  56            push    esi                              hWnd
00401BC0  .  FF15 28F86F0  call    dword ptr [<&USER32.SendDlgItem  SendDlgItemMessageW
00401BC6  .  8B2D 0CF86F0  mov     ebp, dword ptr [<&USER32.Enable  USER32.EnableWindow
```

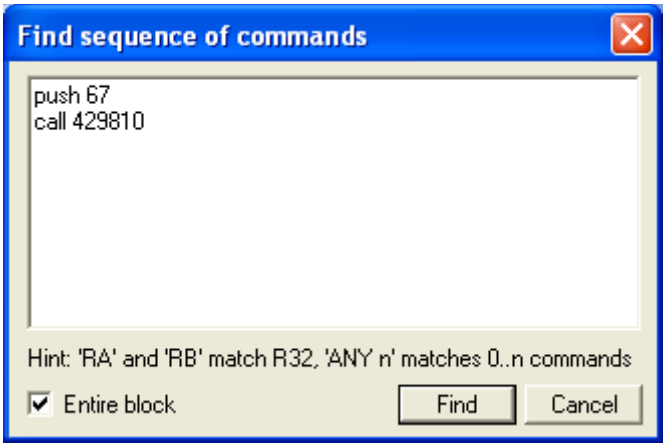
La llamada es del siguiente tipo:

```
push ID_String
call 429810
```

Busquemos el valor de la string que más nos gusta en Resource Hacker:

```
STRINGTABLE
LANGUAGE LANG_ENGLISH, SUBLANG_ENGLISH_UK
{
101,   "%d day remaining."
102,   "%d days remaining."
103,   "You are using a licensed copy of mIRC that is registered to the following indi
105,   "Your evaluation copy of mIRC has expired. To continue using mIRC, please regis
106,   "You are currently using an evaluation copy of mIRC. If you like mIRC, please r
107,   "Exit"
108,   "Continue"
109,   "To register mIRC, click on the register button below."
110,   "If you like mIRC, and use it often, please register your copy."
111,   "To register mIRC, click on the register button below."
}
```

Hagamos una búsqueda en Olly de esta manera:



Desde 'Search for -> All sequences' y encontrará una única dirección:

```
004019D2  .  68 90CD8100   push    0081CD90
004019D7  .  6A 00         push    0x0
004019D9  .  6A 67         push    0x67                    // ** REGISTERED
004019DB  .  E8 307E0200   call    00429810
004019E0  .  83C4 08       add     esp, 0x8
004019E3  .  50            push    eax
004019E4  .  8D4424 38     lea     eax, dword ptr [esp+0x38]
004019E8  .  68 D0FA6F00   push    006FFAD0                UNICODE "%s♪▯♪▯%s"
004019ED  .  50            push    eax
```

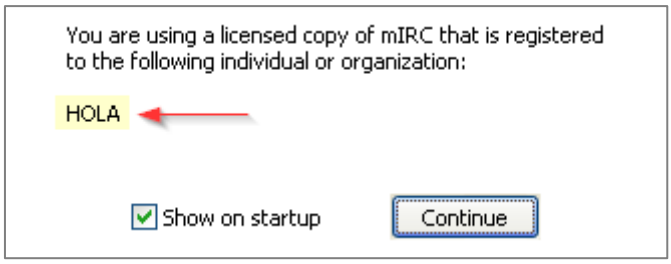
Hacemos un scroll de ratón hacia arriba en el código y encontramos dos posibles saltos que evitan pasar por aquí, vamos al primero de éstos (el de más arriba) y colocamos un BP en él, reiniciamos Olly y lo dejamos correr libremente con <F9> y Olly para:

```
00401800  $  B8 F4200000          mov     eax, 0x20F4
00401805  .  E8 46CF1B00          call    005BE750
0040180A  .  A1 888A7800          mov     eax, dword ptr [0x788A88]
0040180F  .  33C4                 xor     eax, esp
00401811  .  898424 F0200000      mov     dword ptr [esp+0x20F0], eax
00401818  .  66:833D 90CD8100 00  cmp     word ptr [0x81CD90], 0x0
00401820  .  8B8424 04210000      mov     eax, dword ptr [esp+0x2104]
00401827  .  53                   push    ebx
00401828  .  8B9C24 04210000      mov     ebx, dword ptr [esp+0x2104]
0040182F  .  55                   push    ebp
00401830  .  56                   push    esi
00401831  .  8BB424 04210000      mov     esi, dword ptr [esp+0x2104]
00401838  .  57                   push    edi
00401839  .v 0F84 26030000        je      00401B65                // ** Unregistered
0040183F  .  833D 00307800 00     cmp     dword ptr [0x783000], 0x0
00401846  .v 0F84 39050000        je      00401D85
0040184C  .  C705 00307800 000000 mov     dword ptr [0x783000], 0x0
00401856  .  85C0                 test    eax, eax
00401858  .v 74 08                je      short 00401862
ds:[0081CD90]=0000
```

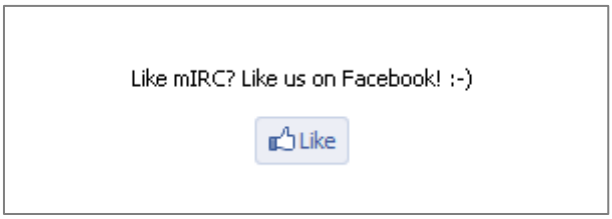
Ese flag está a 0x0, pongamos algo en él:

```
Address   Hex dump                                         ASCII
0081CD90  48 00 4F 00 4C 00 41 00 00 00 00 00 00 00 00 00  H.O.L.A.........
0081CDA0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0081CDB0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0081CDC0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0081CDD0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0081CDE0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0081CDF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0081CE00  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0081CE10  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0081CE20  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Dejamos correr Olly con <F9> y nos llevamos una grata sorpresa:



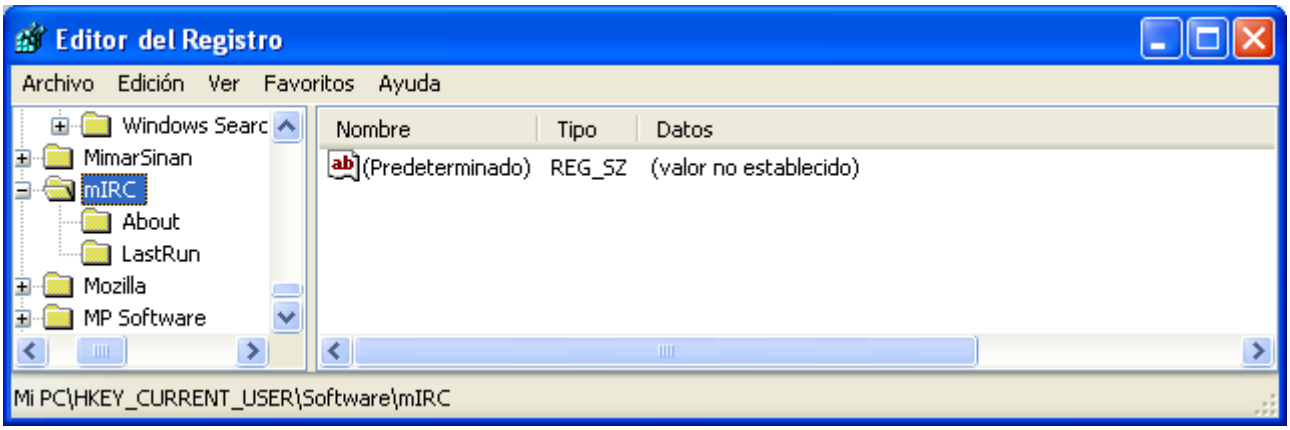
Si tocamos la ventana About vemos que el programa realmente 'cree' que estamos registrados:



Esto es del About.

Y no hemos tocado absolutamente nada, de hecho si adelantamos el reloj 1 año el programa arranca EXPIRED, sin embargo, colocando alguna String en 0x81CD90 saltas toda la protección y corre libre.

Si miramos el registro vemos que ni si quiera existen las claves:



Desde aquí se abriría una nueva línea de ataque, independientemente a la anterior que recorrimos en el punto 7. Es habitual, muy habitual encontrar distintas líneas para atacar una protección, algunas con más trabajo que otras.

Podemos preparar un archivo de registro *.reg que las escriba por nosotros:

```
Windows Registry Editor Version 5.00

[HKEY_CURRENT_USER\Software\mIRC]

[HKEY_CURRENT_USER\Software\mIRC\About]
@="0,1"

[HKEY_CURRENT_USER\Software\mIRC\LastRun]
@="1457084458,1457084963"

[HKEY_CURRENT_USER\Software\mIRC\License]
@="6555-593223"

[HKEY_CURRENT_USER\Software\mIRC\UserName]
@="Snat[CLS]"

[HKEY_CURRENT_USER\Software\mIRC\Validated]
@="2731c26fbc02CAA7A1cE34A33bB1be830B2694e5eda6a986121212redirect=7878"
```

Lo salvamos como registro.reg y con un doble clic de ratón escribiría las claves por nosotros.

También podríamos inyectar una pequeña rutina que escribiera nuestro nombre en 0x81CD90 y cree las claves antes de ejecutar el programa principal.

Como se puede ver, las opciones son muchísimas, a gusto del consumidor :)

8. Prueba de fuego

Sólo nos queda probar el trabajo en Windows 7, en otra máquina distinta y con ASLR, vamos a confirmar la teoría, recuperamos el archivo mirt_final.exe de la parte 6 de la papelera de reciclaje y lo movemos cargamos en Olly (en Win7), calculemos la dirección de cualquier modificación que hicimos:

```
005021C5 . /0F84 6C010000       je  00502337              ;  // ** NO_REVALIDA
```

Por ejemplo esta, está tomada de XP con ImageBase 0x400000, su offset relativo será:

```
offset = 0x5021C5 - 0x400000 == 1021C5
```

En en entry point (en Win7) miramos el valor del ImageBase para esta ejecución:

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped as
006D0000	00003000				Priv	RW	RW	
006E0000	0000C000				Map	R	R	
00860000	00003000				Map	R	R	
00870000	00181000				Map	R	R	
01130000	00001000	mirc_fin		PE header	Imag	R	RWE	
01131000	002FE000	mirc_fin	.text	SFX,code	Imag	R	RWE	
0142F000	00084000	mirc_fin	.rdata	data,import	Imag	R	RWE	
014B3000	001EB000	mirc_fin	.data		Imag	R	RWE	
0169E000	000A7000	mirc_fin	.rsrc	resources	Imag	R	RWE	
01745000	0002E000	mirc_fin	.reloc		Imag	R	RWE	
01780000	000C7000				Map	R	R	

Está en 0x1130000, calculemos la dirección:

```
dirección = 0x1130000 + 1021C5
          = 0x12321C5
```

Vayamos hasta allí:

```
012321BA    57                  push    edi
012321BB    E8 B0D70900         call    012CF970
012321C0    83C4 0C             add     esp, 0xC
012321C3    3BC3                cmp     eax, ebx
012321C5    E9 6D010000         jmp     01232337
012321CA    90                  nop
012321CB    8D78 12             lea     edi, dword ptr [eax+0x12]
012321CE    0FB707              movzx   eax, word ptr [edi]
012321D1    66:3BC3             cmp     ax, bx
012321D4    0F84 03010000       je      012322DD
012321DA    66:83F8 22          cmp     ax, 0x22
012321DE    75 19               jnz     short 012321F9
```

El JMP que ensamblamos en 0x5021C5, confirmado que el ASLR no es molestia, pulsamos <F9> y el programa arranca perfectamente.

Como observación final decir que la ventana de About salva su estado (mostrarse o no mostrarse al inicio) en el siguiente archivo *.ini

- ✓ C:\Documents and Settings\snat\Datos de programa\mIRC\mirc.ini

Siendo snat el usuario actual, en su interior está bajo la clave:

```
[about]
portable=no
version=7.43
show=yes          // ** Podrá ser NO según convenga !
```

9. Notas Finales

Una protección fácil que nos permite practicar métodos básicos dentro del mundo del reversing: Localizar la función de respuesta al evento de un botón (que es una ventana), tracear el código de arriba abajo y de abajo a arriba (sentido inverso), localización de strings en la STRING_TABLE y posterior carga de éstas mediante LoadStringA/W, control de integridad del ejecutable, etc..

En fin, un tema divertido que nos ha hecho pasar el rato. Me gustaría dedicar el texto a Juan Fary que me pasó un enlace al instalador.

Asimismo enviar un fuerte abrazo a Ricardo Narvaja hoy que es su cumpleaños ¡Felicidades!

Por su puesto un saludo a todo el grupo de CracksLatinoS !

Dicho esto nos vamos a exploitear todo lo que podamos en el canal CLS Exploits donde estamos aprendiendo y practicando el noble arte de Exploitear, si te gusta y te animas este es el enlace:

https://telegram.me/joinchat/Bz3s0gge4qUyuF3DkBxYMw

Follow the link and join us !

Hasta la próxima..
Snat.~